

Schema Registry integration with YangKit

INSA de Lyon

Alex Huang Feng (alex.huang-feng@insa-lyon.fr)

Vivek Boudia (vivekananda.boudia@insa-lyon.fr)

Pierre Francois (pierre.francois@insa-lyon.fr)

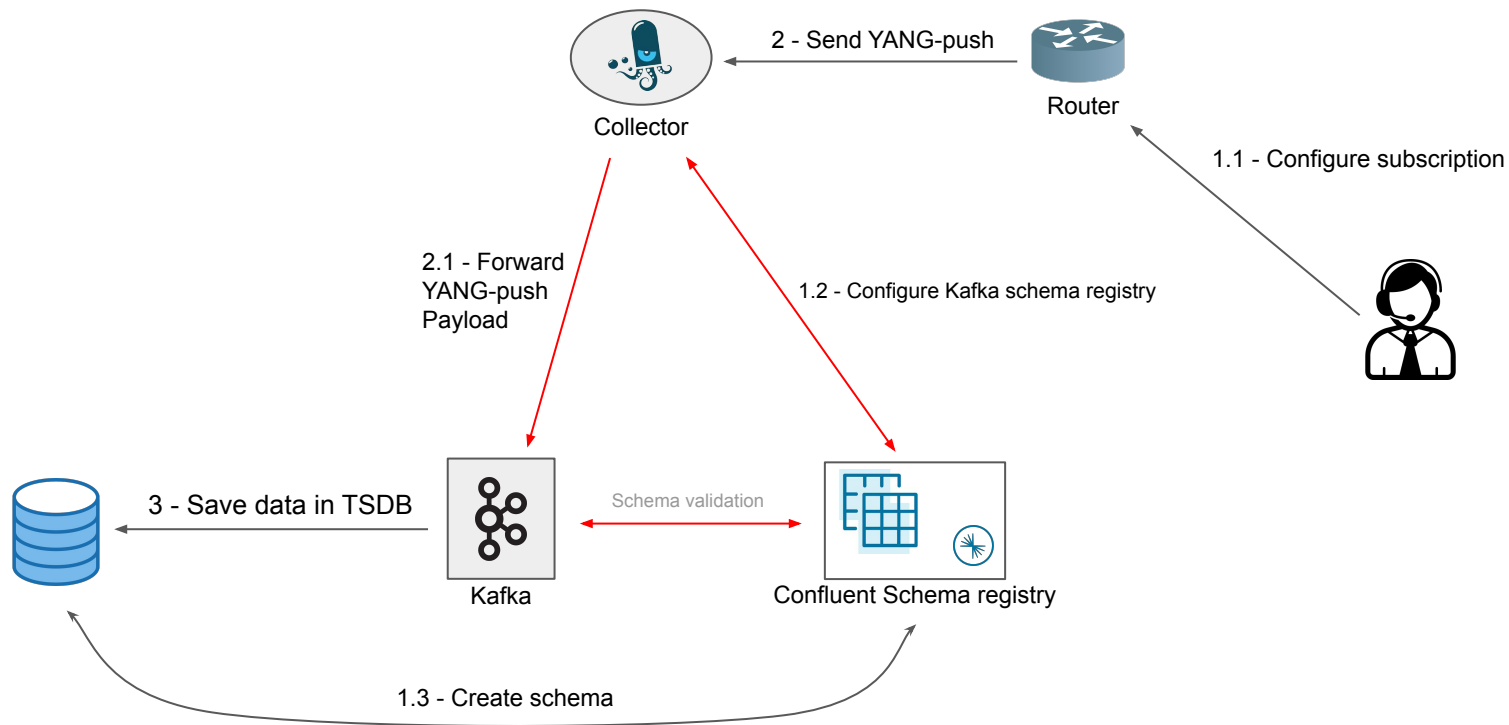
Last Updated: July 20th 2024

Index

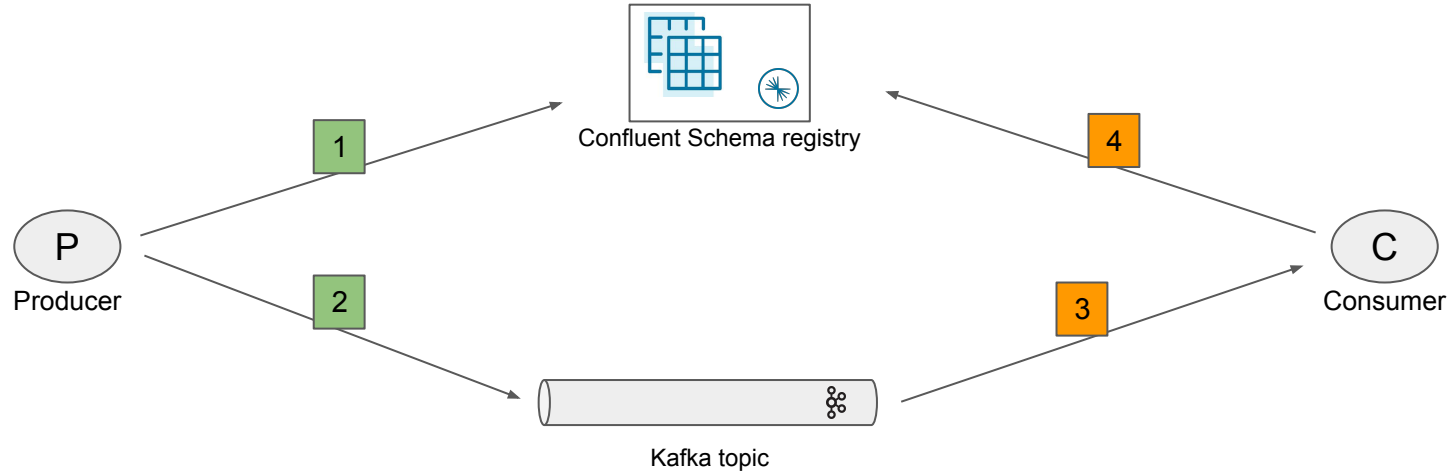
- Schema Registry
 - How does Schema registry works?
 - How YANG push modules are structured
 - Design choices for YANG integration
 - Status
- YangKit
 - Interface with YangKit (JSON and CBOR)
 - Content encapsulated in “data” node
 - YANG data validation
 - YANG push Notification validation
- Missing pieces: libyangserde (C library adding the MAGIC BYTE and the schema Id into the Kafka message)

Schema Registry

Full Implementation-level architecture

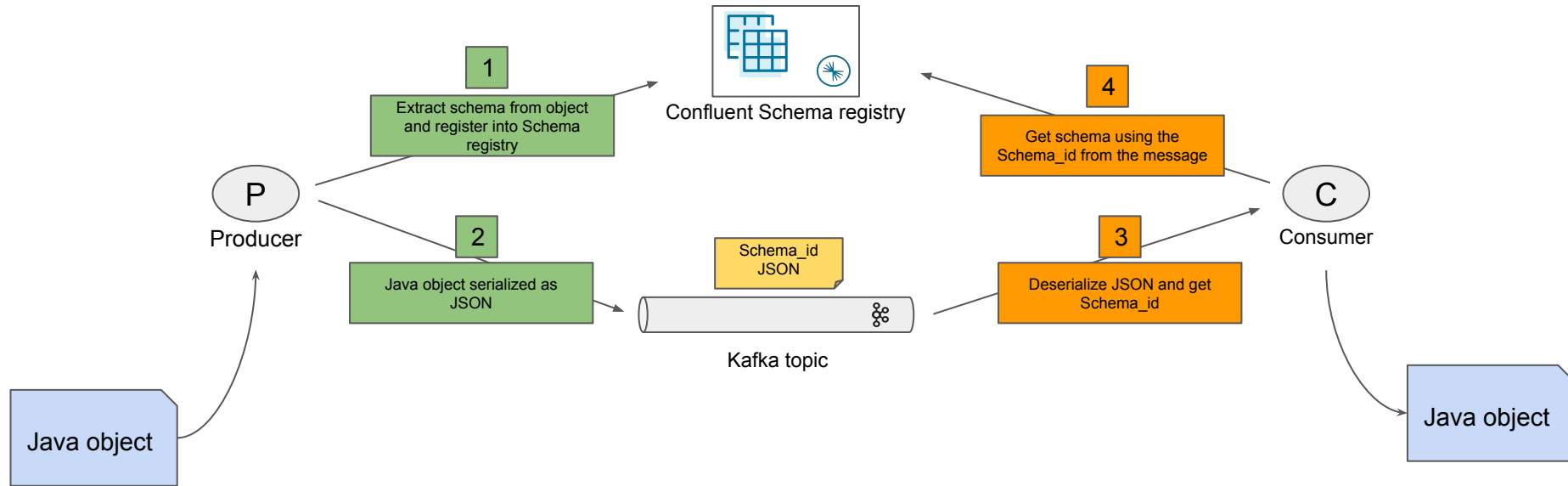


How does Schema registry works?

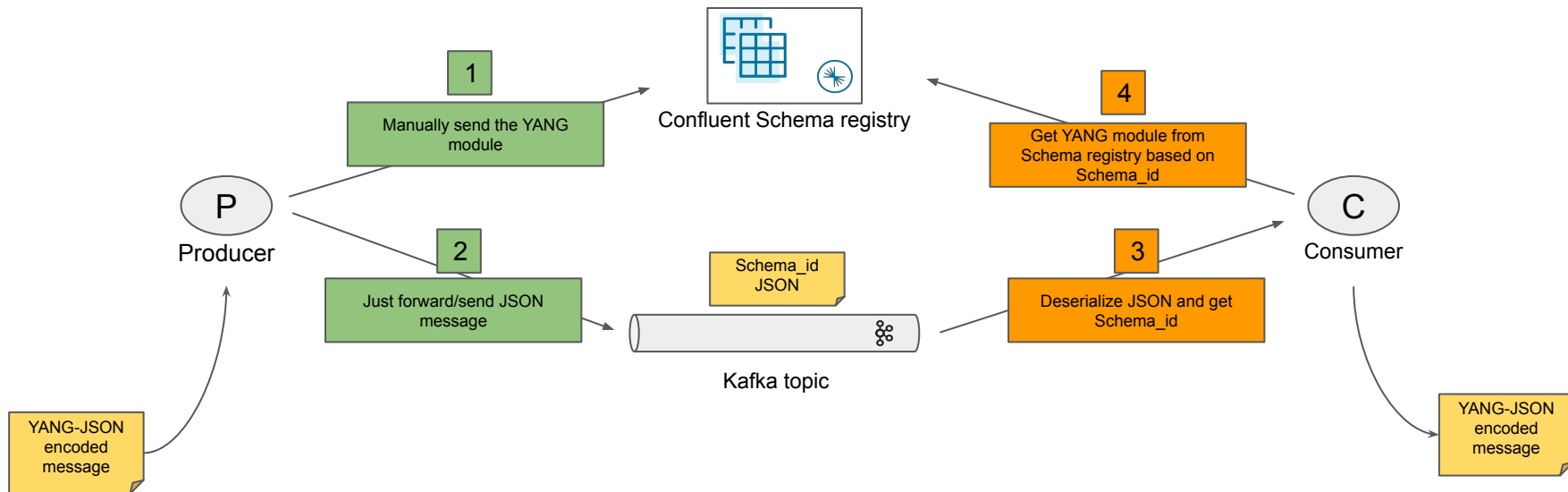


1. register schema
2. Send <MAGICBYTE, Schema ID, Content>
3. Receive <MAGICBYTE, Schema ID, Content>
4. Get schema

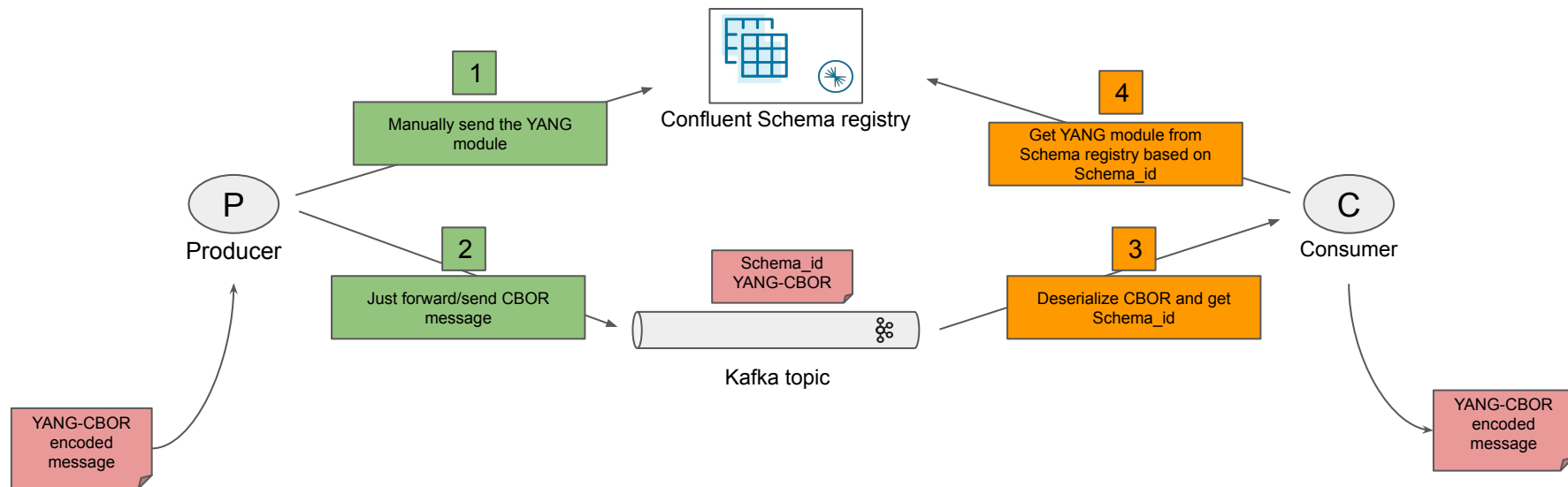
Main usecase for Schema registry: Plain Old Java Objects



Main usecase for YANG: Using YANG-JSON (without POJO)



Main usecase for YANG: Using YANG-CBOR (without POJO)



Same as YANG-JSON encoding but in YANG-CBOR

Nature of YANG modules / schemas

Example:

```
module insa-container {  
  yang-version 1.1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-container";  
  prefix ic;  
  
  container first-container {  
    config false;  
  
    leaf address {  
      type string;  
    }  
    leaf port {  
      type uint8;  
    }  
  }  
}
```

```
module insa-augment {  
  yang-version 1.1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-augment";  
  prefix ia;  
  
  import insa-container {  
    prefix ic;  
    reference "RFC XXXX: YYYY";  
  }  
  
  augment "/ic:first-container" {  
    leaf mtu {  
      type string;  
      config false;  
    }  
    container second-container {  
      config false;  
      leaf identifier {  
        type string;  
      }  
    }  
  }  
}
```

```
module insa-augment-bis {  
  yang-version 1.1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-augment-bis";  
  prefix iab;  
  
  import insa-container {  
    prefix ic;  
    reference "RFC XXXX: YYYY";  
  }  
  import insa-augment {  
    prefix ia;  
    reference "RFC XXXX: YYYY";  
  }  
  
  augment "/ic:first-container/ia:second-container" {  
    leaf name {  
      type string;  
      config false;  
    }  
  }  
}
```

Nature of YANG modules / schemas → Dependencies

```
module insa-container {  
  yang-version 1.1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-container";  
  prefix ic;  
  
  container first-container {  
    config false;  
  
    leaf address {  
      type string;  
    }  
    leaf port {  
      type uint8;  
    }  
  }  
}
```

```
module insa-augment {  
  yang-version 1.1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-augment";  
  prefix ia;  
  
  import insa-container {  
    prefix ic;  
    reference "RFC XXXX: YYYY";  
  }  
  
  augment "/ic:first-container" {  
    leaf mtu {  
      type string;  
      config false;  
    }  
    container second-container {  
      config false;  
      leaf identifier {  
        type string;  
      }  
    }  
  }  
}
```

```
module insa-augment-bis {  
  yang-version 1.1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-augment-bis";  
  prefix iab;  
  
  import insa-container {  
    prefix ic;  
    reference "RFC XXXX: YYYY";  
  }  
  import insa-augment {  
    prefix ia;  
    reference "RFC XXXX: YYYY";  
  }  
  
  augment "/ic:first-container/ia:second-container" {  
    leaf name {  
      type string;  
      config false;  
    }  
  }  
}
```

insa-container.yang ← insa-augment.yang ← insa-augment-bis.yang

Nature of YANG modules / schemas: registration in Schema registry

insa-container.yang ← insa-augment.yang ← insa-augment-bis.yang

1

2

3

1. Register insa-container.yang (subject **insa-container**)
2. Register insa-augment.yang (subject **insa-augment**)
3. Register insa-augment-bis.yang (subject **insa-augment-bis**)
4. Use **last subject*** (*insa-augment-bis*) to validate message

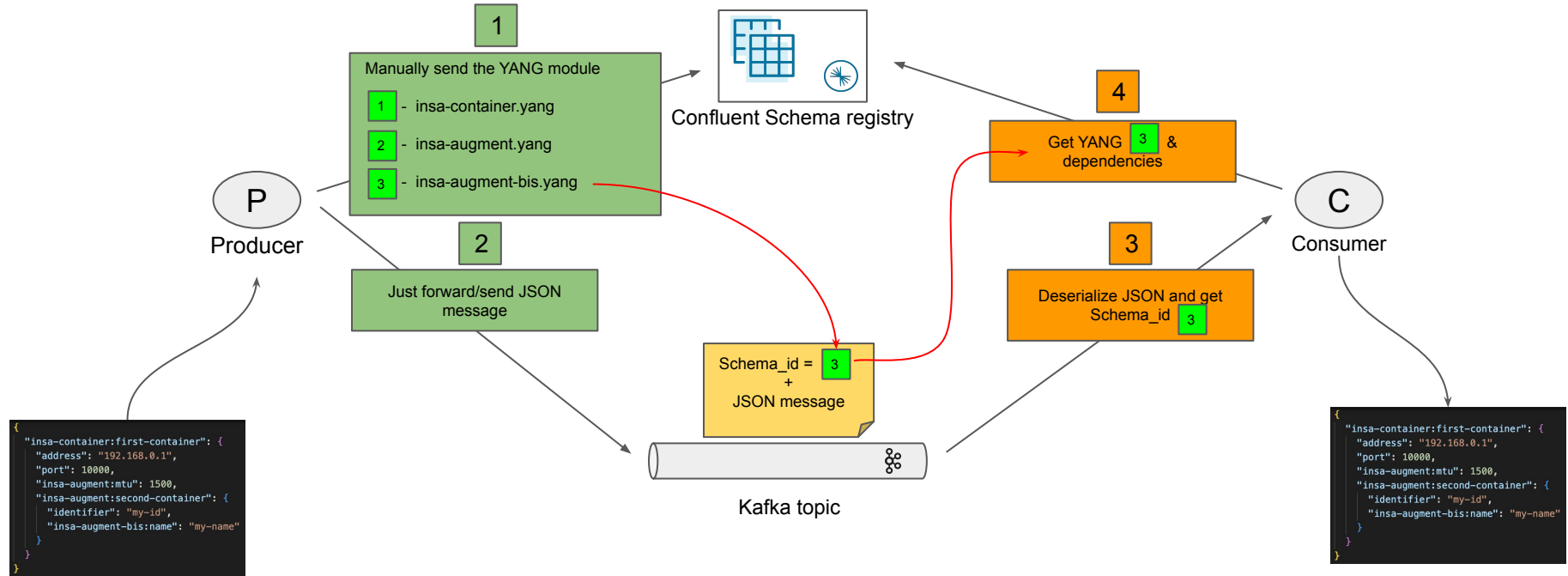
```
module: insa-container
  +-ro first-container
    +-ro address?      string
    +-ro port?         uint8
    +-ro ia:mtu?       string
    +-ro ia:second-container
      +-ro ia:identifier? string
      +-ro iab:name?    string
```



```
{
  "insa-container:first-container": {
    "address": "192.168.0.1",
    "port": 10000,
    "insa-augment:mtu": 1500,
    "insa-augment:second-container": {
      "identifier": "my-id",
      "insa-augment-bis:name": "my-name"
    }
  }
}
```

*Schema registry Subject: Identifier of the schema used for data validation.

Main usecase for YANG: Using YANG-JSON (without POJO)



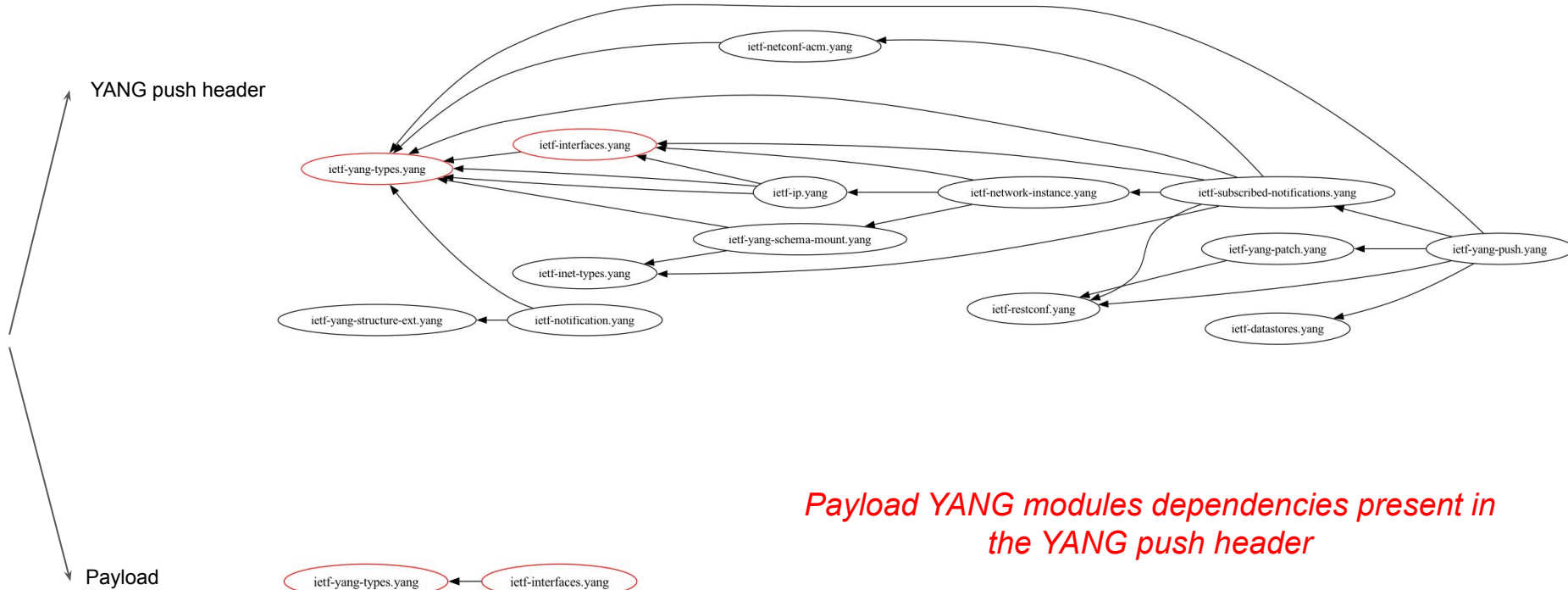
YANG push in Schema registry: **example 1**

```
{
  "ietf-notification:notification": {
    "eventTime": "2023-03-25T08:30:11.22Z",
    "ietf-notification-sequencing:sysName": "example-router",
    "ietf-notification-sequencing:sequenceNumber": 1,
    "ietf-yang-push:push-update": {
      "id": 6666,
      "ietf-yang-push-netobs-timestamping:observation-time": "2023-03-25T08:30:11.22Z",
      "datastore-contents": {
        "ietf-interfaces:interfaces": [
          {
            "interface": {
              "name": "eth0",
              "type": "iana-if-type:ethernetCsmacd",
              "oper-status": "up"
            }
          }
        ]
      }
    }
  }
}
```

YANG-push header

YANG-push payload

YANG push in Schema registry: dependencies



YANG push in Schema registry: **example 2**

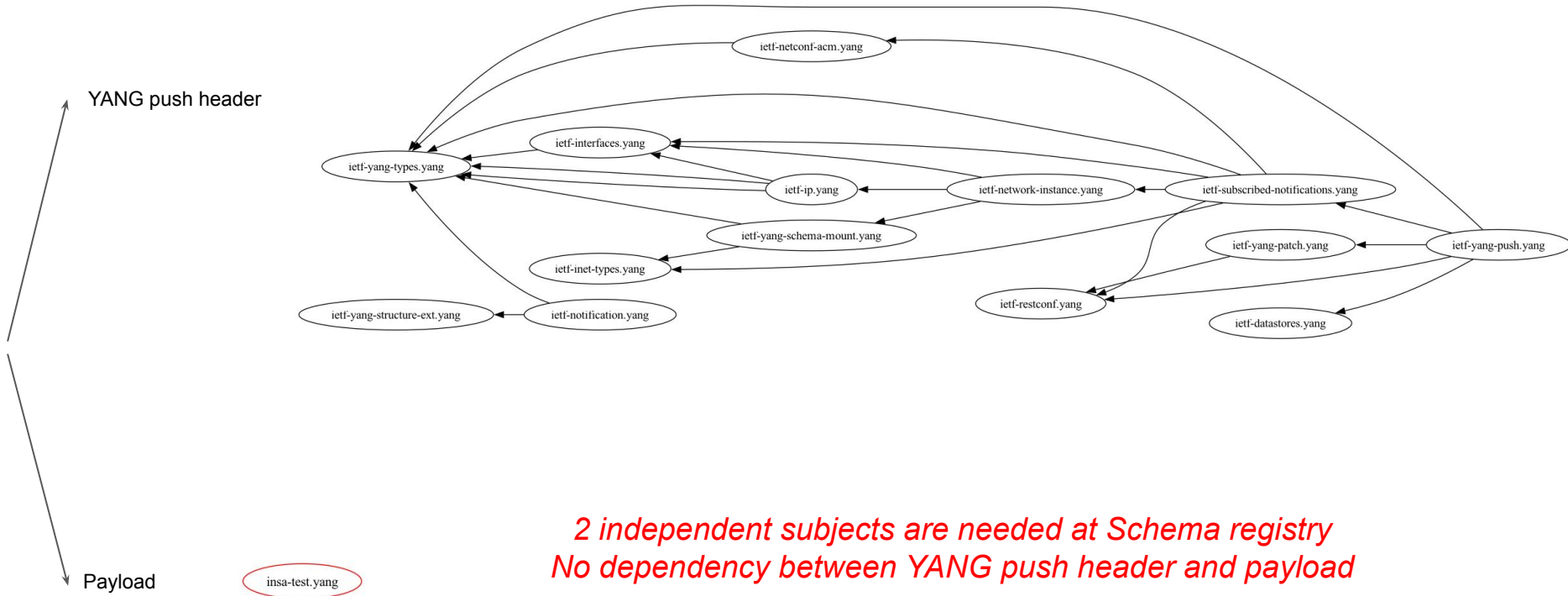
```
{
  "ietf-notification:notification": {
    "eventTime": "2023-03-25T08:30:11.22Z",
    "ietf-notification-sequencing:sysName": "example-router",
    "ietf-notification-sequencing:sequenceNumber": 1,
    "ietf-yang-push:push-update": {
      "id": 6666,
      "ietf-yang-push-netobs-timestamping:observation-time": "2023-03-25T08:30:11.22Z",
      "datastore-contents": {
        "insa-test:insa-container": {
          "computer": "my-computer",
          "router": 26
        }
      }
    }
  }
}
```

YANG-push header

Independent YANG

Note: *insa-test.yang* is a non-standard YANG module for this example. Same usecase as if OpenConfig YANG modules were used.

YANG push in Schema registry: dependencies



Complexity coming from the IETF Specification

RFC8641: YANG push

notifications:

```
+---n push-update
|   +---ro id?                sn:subscription-id
|   +---ro datastore-contents? <anydata>
|   +---ro incomplete-update? empty
+---n push-change-update {on-change}?
    +---ro id?                sn:subscription-id
    +---ro datastore-changes
    |   +---ro yang-patch
    |   |   +---ro patch-id    string
    |   |   +---ro comment?    string
    |   |   +---ro edit* [edit-id]
    |   |       +---ro edit-id    string
    |   |       +---ro operation enumeration
    |   |       +---ro target    target-resource-offset
    |   |       +---ro point?    target-resource-offset
    |   |       +---ro where?    enumeration
    |   |       +---ro value?    <anydata>
    |   +---ro incomplete-update? empty
```

RFC7950: YANG 1.1

7.10. The "anydata" Statement

The "anydata" statement defines an interior node in the schema tree. It takes one argument, which is an identifier, followed by a block of substatements that holds detailed anydata information.

The "anydata" statement is used to represent an unknown set of nodes that can be modeled with YANG, except anyxml, but for which the data model is not known at module design time. It is possible, though not required, for the data model for anydata content to become known through protocol signaling or other means that are outside the scope of this document.

An example of where anydata can be useful is a list of received notifications where the specific notifications are not known at design time.

An anydata node cannot be augmented (see [Section 7.17](#)).

An anydata node exists in zero or one instance in the data tree.

An implementation may or may not know the data model used to model a specific instance of an anydata node.

Since the use of anydata limits the manipulation of the content, the "anydata" statement SHOULD NOT be used to define configuration data.

Designs for integrating YANG to Schema registry

- (1) Create a YANG module integrating all the YANG push header and payload
 - First approach proposed at Hackathon 117
 - **Issue**: Creating a new YANG module changes the namespace of the encoded message.

- (2) Register all the YANG modules to a Schema Context*
 - Approach taken by YangKit (currently testing this approach at INSA)
 - **Issue**: Schema registry needs to support “one subject is associated to multiple models” (not only augmentations, but independent modules)

*Schema Context: Instance in memory registering all YANG modules for data validation (of a message)

Design 1: Craft a new YANG module based on the modules coming from the routers

ietf-inet-types@2021-02-22.yang
ietf-yang-types@2023-01-23.yang
ietf-restconf@2017-01-26.yang
ietf-datastores@2018-02-14.yang
insa-test@2023-09-05.yang
ietf-yang-structure-ext@2020-06-17.yang
ietf-interfaces@2018-02-20.yang
ietf-yang-patch@2017-02-22.yang
ietf-netconf-acm@2018-02-14.yang
ietf-yang-schema-mount@2019-01-14.yang
ietf-ip@2018-02-22.yang
ietf-network-instance@2019-01-21.yang
ietf-notification@2023-07-23.yang
ietf-subscribed-notifications@2019-09-09.yang
ietf-yang-push@2019-09-09.yang

crafted-yang-push@2023-11-07.yang

*Includes all leaves and types defined in the set of YANG modules

Observations:

- the YANG modules from the routers are not used anymore once the crafted YANG is created
- Filtering (datatree/Xpath) can be managed directly from the YANG module

Design 1: Craft a new YANG module based on the modules coming from the routers

crafted-yang-push@2023-11-07.yang ← Validate message →

```
{
  "ietf-notification:notification": {
    "eventTime": "2023-03-25T08:30:11.22Z",
    "ietf-notification-sequencing:sysName": "example-router",
    "ietf-notification-sequencing:sequenceNumber": 1,
    "ietf-yang-push:push-update": {
      "id": 6666,
      "ietf-yang-push-netobs-timestamping:observation-time": "2023-03-25T08:30:11.22Z",
      "datastore-contents": {
        "insa-test:insa-container": {
          "computer": "my-computer",
          "router": 26
        }
      }
    }
  }
}
```

Validation happens directly against a single YANG module

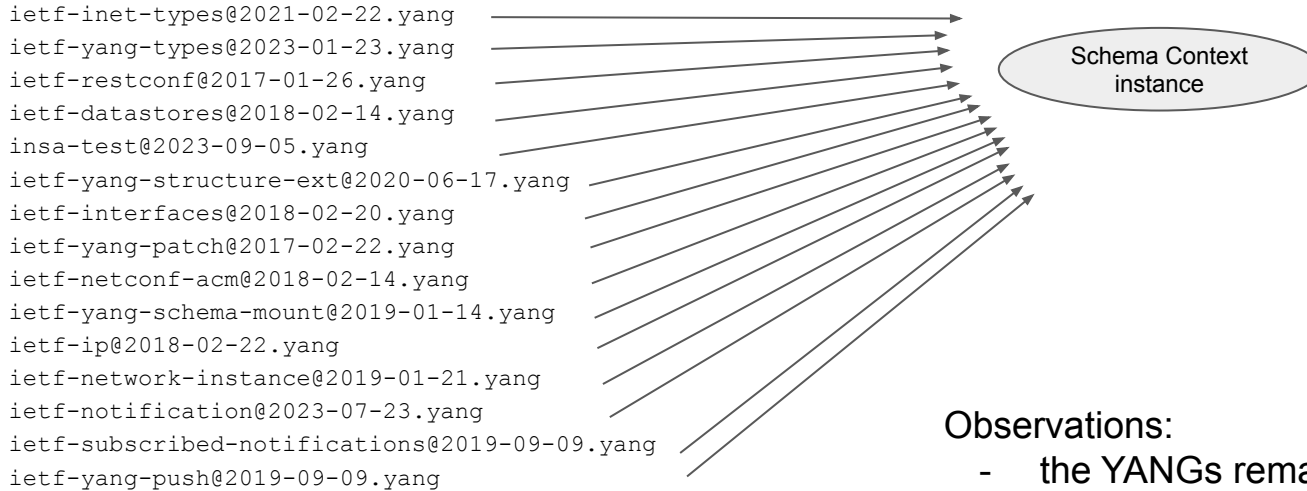
Designs for integrating YANG to Schema registry

- (1) Create a YANG module integrating all the YANG push header and payload
 - First approach proposed at Hackathon 117
 - **Issue**: Creating a new YANG module changes the namespace of the encoded message.

- (2) Register all the YANG modules to a Schema Context*
 - Approach taken by YangKit (currently testing this approach at INSA)
 - **Issue**: Schema registry needs to support “one subject is associated to multiple models” (not only augmentations, but independent modules)

**Schema Context: Instance in memory registering all YANG modules for data validation (of a message)*

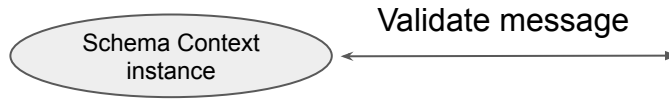
Design 2: Register all YANG modules to an Schema context



Observations:

- the YANGs remain intact from the routers
- YANG push message need to be validated against a Schema context, not a schema as defined in Schema registry

Design 2: Register all YANG modules to an Schema context



```
{
  "ietf-notification:notification": {
    "eventTime": "2023-03-25T08:30:11.22Z",
    "ietf-notification-sequencing:sysName": "example-router",
    "ietf-notification-sequencing:sequenceNumber": 1,
    "ietf-yang-push:push-update": {
      "id": 6666,
      "ietf-yang-push-netobs-timestamping:observation-time": "2023-03-25T08:30:11.22Z",
      "datastore-contents": {
        "insa-test:insa-container": {
          "computer": "my-computer",
          "router": 26
        }
      }
    }
  }
}
```

Schema context needs to be instantiated on a message version basis

Designs for integrating YANG to Schema registry

- (1) Create a YANG module integrating all the YANG push header and payload
 - First approach proposed at Hackathon 117
 - **Issue**: Creating a new YANG module changes the namespace of the encoded message.

- (2) Register all the YANG modules to a Schema Context*
 - Approach taken by YangKit (currently testing this approach at INSA)
 - **Issue**: Schema registry needs to support “one subject is associated to multiple models” (not only augmentations, but independent modules)

**Schema Context: Instance in memory registering all YANG modules for data validation (of a message)*

Complexity coming from the IETF Specification

RFC8641: YANG push

notifications:

```
+---n push-update
|   +---ro id?                sn:subscription-id
|   +---ro datastore-contents? <anydata>
|   +---ro incomplete-update? empty
+---n push-change-update {on-change}?
    +---ro id?                sn:subscription-id
    +---ro datastore-changes
    |   +---ro yang-patch
    |   |   +---ro patch-id    string
    |   |   +---ro comment?    string
    |   |   +---ro edit* [edit-id]
    |   |       +---ro edit-id    string
    |   |       +---ro operation enumeration
    |   |       +---ro target    target-resource-offset
    |   |       +---ro point?    target-resource-offset
    |   |       +---ro where?    enumeration
    |   |       +---ro value?    <anydata>
    |   +---ro incomplete-update? empty
```

RFC7950: YANG 1.1

7.10. The "anydata" Statement

The "anydata" statement defines an interior node in the schema tree. It takes one argument, which is an identifier, followed by a block of substatements that holds detailed anydata information.

The "anydata" statement is used to represent an unknown set of nodes that can be modeled with YANG, except anyxml, but for which the data model is not known at module design time. It is possible, though not required, for the data model for anydata content to become known through protocol signaling or other means that are outside the scope of this document.

An example of where anydata can be useful is a list of received notifications where the specific notifications are not known at design time.

An anydata node cannot be augmented (see [Section 7.17](#)).

An anydata node exists in zero or one instance in the data tree.

An implementation may or may not know the data model used to model a specific instance of an anydata node.

Since the use of anydata limits the manipulation of the content, the "anydata" statement SHOULD NOT be used to define configuration data.

Impact on Schema registry using Design (2)

Register all the YANG modules to a Schema Context

- **Issue:** Schema registry needs to support “one subject is associated to multiple models” (not only augmentations, but independent modules)
 - Solution 1: Implement support in schema registry
 - API is changed and not “compliant” to Confluent approach (meaning we are implementing a new API)
 - Solution 2: (workaround) Global SchemaContext per Schema Registry
 - API is not changed, but upon request, the global SchemaContext is used
 - YANG versioning and BC/NBC checks cannot be supported

Yangkit

Interfaces

Gaps

Interface with YangKit (JSON)

Input:

- JsonNode (Jackson library)
- SchemaContext (Class having all Serialised yang modules)

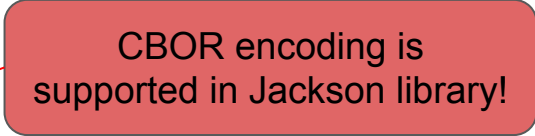
Output:

- Notification is valid/invalid
- Data is valid/invalid

Interface with YangKit (CBOR)

Input:

- JsonNode (Jackson library)
- SchemaContext (Class having all Serialised yang modules)



CBOR encoding is supported in Jackson library!

Output:

- Notification is valid/invalid
- Data is valid/invalid

Validating YANG-JSON **data**

YANG data is wrapped in “data” node

- *Is it a Standard/expected behavior?*
- *Behavior coming from NETCONF RFC6241?*
- Easy to change

```
{
  "data": {
    "insa-custom:insa-container": {
      "computer": 1,
      "router": 234
    }
  }
}
```

Schema registry - Status (16/05/2024) [Phase 1]

- YangKit integrated to Schema Registry as a PoC
- Encodings
 - YANG JSON
 - YANG CBOR

Demo - YANG module

```
module insa-custom {  
  yang-version 1;  
  namespace "urn:ietf:params:xml:ns:yang:insa-yang";  
  prefix iny;  
  
  contact "alex.huang-feng@insa-lyon.fr";  
  organization "INSA Lyon";  
  description "insa-custom yang module";  
  
  revision 2019-09-09 {  
    description "Initial version.";  
  }  
  
  container insa-container {  
    config false;  
    leaf computer {  
      type string;  
      description "computer";  
    }  
    leaf router {  
      type uint8;  
      description "router";  
    }  
  }  
}
```

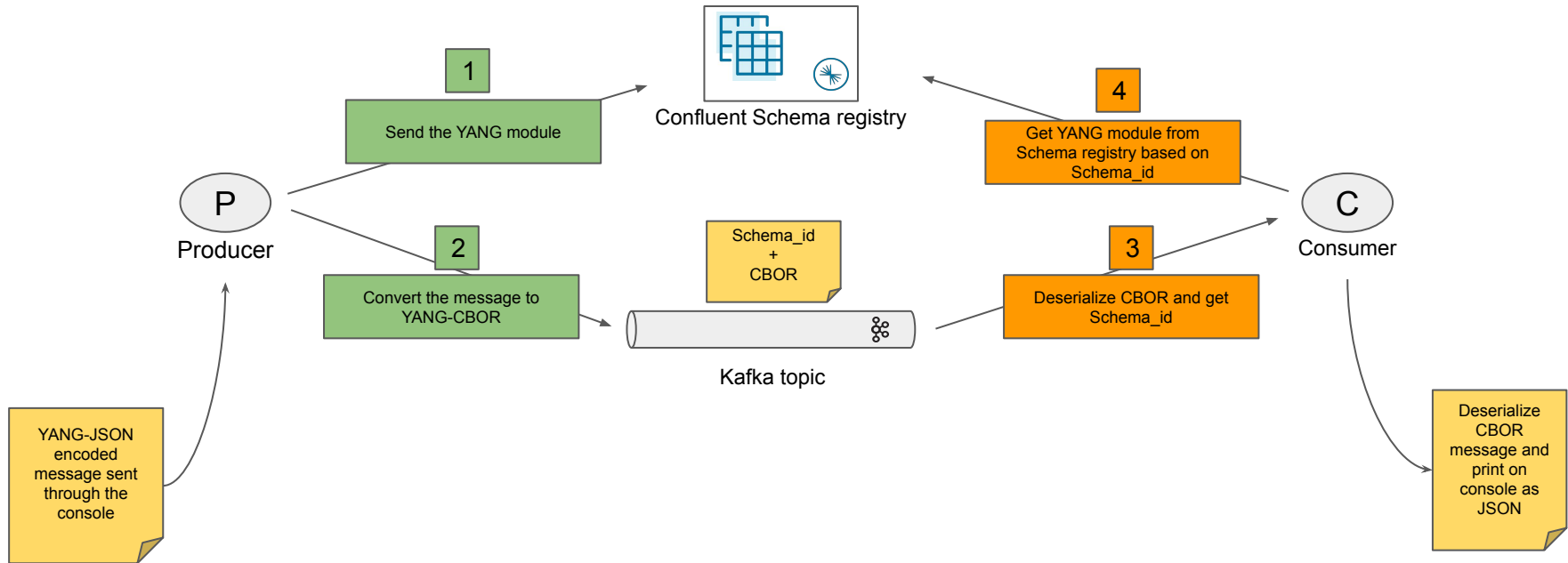
ideally

```
{  
  "insa-custom:insa-container": {  
    "computer": "computer",  
    "router": 234  
  }  
}
```

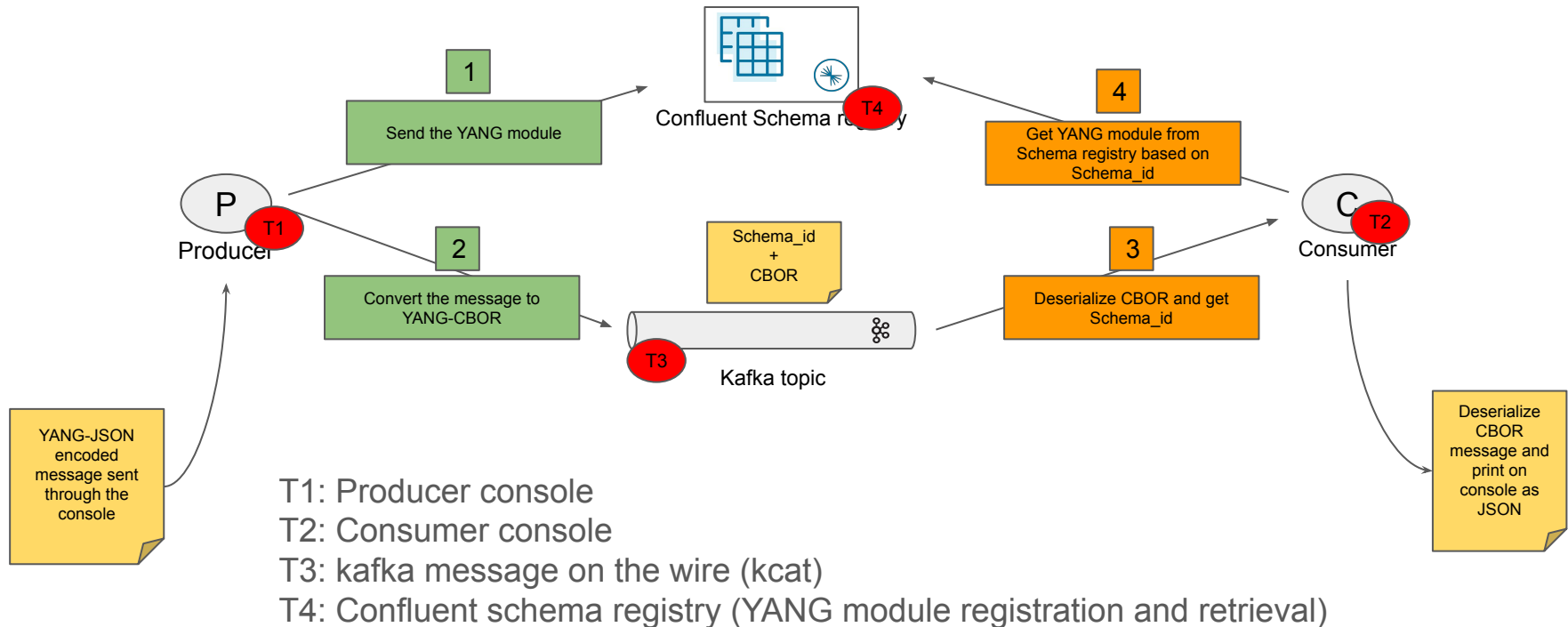
wrapped due to current
YangKit implementation

```
{  
  "data": {  
    "insa-custom:insa-container": {  
      "computer": "computer",  
      "router": 234  
    }  
  }  
}
```


What does it do?



What does it do?



How to test → use schema registry console producer

Producer

T1: producer

```
Apple > ~/Unyte/shared/kafka-mesh/schema-registry-dev > feature/yang-serializer !32 INT * < 20m 2s X < 10:56:40
./bin/kafka-yang-cbor-schema-console-producer --bootstrap-server localhost:9092 --property schema.registry.url=http://localhost:8081 --topic yang.tests --property value.schema='module insa-custom {yang-version 1;namespace "urn:ietf:params:xml:ns:yang:insa-yang";prefix iny;contact "alex.huang-feng@insa-lyon.fr";organization "INSA Lyon";description "insa-custom yang module";revision 2019-09-09 {description "Initial version.";}container insa-container {config false;leaf computer {type string;description "computer";}leaf router {type uint8;description "router";}}}'
```

Consumer

T2: consumer

```
Apple > ~/Unyte/shared/kafka-mesh/schema-registry-dev > yangdatadoc ✓ < 23:32:00
./bin/kafka-yang-cbor-schema-console-consumer --bootstrap-server localhost:9092 --from-beginning --topic yang.tests --property schema.registry.url=http://localhost:8081
```

After analysis, the Consumer uses the YANG from schema registry, not the one in the property parameter.

```
./bin/kafka-yang-cbor-schema-console-producer --bootstrap-server --property
schema.registry.ssl.cipher.suites = null
schema.registry.ssl.enabled.protocols = [TLSv1.2, TLSv1.3]
schema.registry.ssl.endpoint.identification.algorithm = https
schema.registry.ssl.engine.factory.class = null
schema.registry.ssl.key.password = null
schema.registry.ssl.keymanager.algorithm = SunX509
schema.registry.ssl.keystore.certificate.chain = null
schema.registry.ssl.keystore.key = null
schema.registry.ssl.keystore.location = null
schema.registry.ssl.keystore.password = null
schema.registry.ssl.keystore.type = JKS
schema.registry.ssl.protocol = TLSv1.3
schema.registry.ssl.provider = null
schema.registry.ssl.secure.random.implementation = null
schema.registry.ssl.trustmanager.algorithm = PKIX
schema.registry.ssl.truststore.certificates = null
schema.registry.ssl.truststore.location = null
schema.registry.ssl.truststore.password = null
schema.registry.ssl.truststore.type = JKS
schema.registry.url = [http://localhost:8081]
use.latest.version = false
use.latest.with.metadata = null
use.schema.id = -1
value.subject.name.strategy = class io.confluent.kafka.serializers.subject.TopicNameStrategy
yang.cbor.fail.invalid.schema = true
yang.cbor.fail.unknown.properties = true
(com.insa.kafka.serializers.yang.cbor.KafkaYangCborSchemaSerializerConfig:369)
```

T1: producer

```
./bin/kafka-yang-cbor-schema-console-consumer --bootstrap-server --topic
schema.registry.ssl.cipher.suites = null
schema.registry.ssl.enabled.protocols = [TLSv1.2, TLSv1.3]
schema.registry.ssl.endpoint.identification.algorithm = https
schema.registry.ssl.engine.factory.class = null
schema.registry.ssl.key.password = null
schema.registry.ssl.keymanager.algorithm = SunX509
schema.registry.ssl.keystore.certificate.chain = null
schema.registry.ssl.keystore.key = null
schema.registry.ssl.keystore.location = null
schema.registry.ssl.keystore.password = null
schema.registry.ssl.keystore.type = JKS
schema.registry.ssl.protocol = TLSv1.3
schema.registry.ssl.provider = null
schema.registry.ssl.secure.random.implementation = null
schema.registry.ssl.trustmanager.algorithm = PKIX
schema.registry.ssl.truststore.certificates = null
schema.registry.ssl.truststore.location = null
schema.registry.ssl.truststore.password = null
schema.registry.ssl.truststore.type = JKS
schema.registry.url = [http://localhost:8081]
use.latest.version = false
use.latest.with.metadata = null
use.schema.id = -1
value.subject.name.strategy = class io.confluent.kafka.serializers.subject.TopicNameStrategy
yang.cbor.fail.invalid.schema = true
yang.cbor.fail.unknown.properties = true
yang.cbor.key.type = class java.lang.Object
yang.cbor.value.type = class java.lang.Object
(com.insa.kafka.serializers.yang.cbor.KafkaYangCborSchemaDeserializerConfig:369)
```

T2: consumer

```
kcatal -b localhost:9092 -t yang.tests -o beginning -C
% Reached end of topic yang.tests [0] at offset 2

hexdump -C file.out
00000000 00 00 00 00 01 bf 64 64 61 74 61 61 bf 78 1c 69 6e |.....ddata.X.in|
00000010 73 61 2d 66 63 75 73 74 6f 6d 3a 09 6e 73 61 2d |isa-fcustom:insa-|
00000020 63 6f 64 61 69 6d 6e 65 72 bf 68 63 6f 6d 70 |confinsfner-hcomp|
00000030 75 74 65 72 68 63 6f 6d 70 75 74 65 72 66 7f |utchercomputerfro|
00000040 75 74 65 72 18 ea ff ff ff 00 00 00 00 01 bf |uter.....|
00000050 64 64 61 74 61 bf 78 1c 69 6e 73 61 2d 66 63 75 |ddata.X.insa-fcu|
00000060 73 74 6f 6d 3a 09 6e 73 61 2d 63 6f 6e 74 61 69 |istom:insa-contai|
00000070 66 6e 65 72 bf 68 63 6f 6d 70 75 74 65 72 68 63 |fner-hcomp|
00000080 6f 6d 70 75 74 65 72 66 72 6f 75 74 65 72 18 ea |computerfouter..|
00000090 ff ff ff 0a |....|
00000094
```

T3: kcat (kafka on the wire)

```
./bin/schema-registry-start config/schema-registry.properties
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.SubjectsResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.SubjectsResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.SchemaResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.SchemaResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.SubjectVersionsResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.SubjectVersionsResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.CompatibilityResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.CompatibilityResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.ModeResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.ModeResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.ServerMetadataResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.ServerMetadataResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.dekregistry.web.rest.resources.DekRegistryResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.dekregistry.web.rest.resources.DekRegistryResource will be ignored.
ERROR StatusLogger Log4j2 could not find a logging implementation. Please add log4j-core to the classpath. Using SimpleLogger to log to the console...
[2024-05-16 11:57:36,271] INFO Started o.e.j.s.ServletContextHandler@9b437f0{/ws,null,AVAILABLE} (org.eclipse.jetty.server.handler.ContextHandler:921)
[2024-05-16 11:57:36,282] INFO Started o.e.j.s.ServletContextHandler@4f0b4272{/ws,null,AVAILABLE} (org.eclipse.jetty.server.handler.ContextHandler:921)
[2024-05-16 11:57:36,297] INFO Started NetworkTrafficServerConnector@9b93fa8[HTTP/1.1, (http/1.1, h2c)](0.0.0.0:8081) (org.eclipse.jetty.server.AbstractConnector:333)
[2024-05-16 11:57:36,298] INFO Started @3694ms (org.eclipse.jetty.server.Server:415)
[2024-05-16 11:57:36,299] INFO Schema Registry version: 7.5.1 commitId: fdede4252ab4927d6cd531418db5e992836a7 (io.confluent.kafka.schema.registry.rest.SchemaRegistryMain:47)
[2024-05-16 11:57:36,299] INFO Server started, listening for requests... (io.confluent.kafka.schema.registry.rest.SchemaRegistryMain:49)
```

T4: Schema registry

partition <pp>, timestamp <ts>.
Timestamp is the number of milliseconds since epoch UTC.
Requires broker >= 0.10.0.0 and librdkafka

```

./bin/kafka-yang-cbor-schema-console-producer --bootstrap-server --property
schema.registry.ssl.keystore.certificate.chain = null
schema.registry.ssl.keystore.key = null
schema.registry.ssl.keystore.location = null
schema.registry.ssl.keystore.password = null
schema.registry.ssl.keystore.type = JKS
schema.registry.ssl.protocol = TLSv1.3
schema.registry.ssl.provider = null
schema.registry.ssl.secure.random.implementation = null
schema.registry.ssl.trustmanager.algorithm = PKIX
schema.registry.ssl.truststore.certificates = null
schema.registry.ssl.truststore.location = null
schema.registry.ssl.truststore.password = null
schema.registry.ssl.truststore.type = JKS
schema.registry.url = [http://localhost:8081]
use.latest.version = false
use.latest.with.metadata = null
use.schema.id = -1
value.subject.name.strategy = class io.confluent.kafka.serializers.subject.TopicNameStrategy
yang.cbor.fail.invalid.schema = true
yang.cbor.fail.unknown.properties = true
(com.insa.kafka.serializers.yang.cbor.KafkaYangCborSchemaSerializerConfig:369)
{"data": {"insa-custom:insa-container": {"computer": "computer", "router": 234}}}
-- YANG SchemaID:1
TODO: ALEX validate jsonNode against Module
Schema Context:1
Modules: insa-custom
Json node: {"data": {"insa-custom:insa-container": {"computer": "computer", "router": 234}}} -- false

```

T1: producer

Sending JSON
message

```

./bin/kafka-yang-cbor-schema-console-consumer --bootstrap-server --topic
schema.registry.ssl.keystore.location = null
schema.registry.ssl.keystore.password = null
schema.registry.ssl.keystore.type = JKS
schema.registry.ssl.protocol = TLSv1.3
schema.registry.ssl.provider = null
schema.registry.ssl.secure.random.implementation = null
schema.registry.ssl.trustmanager.algorithm = PKIX
schema.registry.ssl.truststore.certificates = null
schema.registry.ssl.truststore.location = null
schema.registry.ssl.truststore.password = null
schema.registry.ssl.truststore.type = JKS
schema.registry.url = [http://localhost:8081]
use.latest.version = false
use.latest.with.metadata = null
use.schema.id = -1
value.subject.name.strategy = class io.confluent.kafka.serializers.subject.TopicNameStrategy
yang.cbor.fail.invalid.schema = true
yang.cbor.fail.unknown.properties = true
yang.cbor.key.type = class java.lang.Object
yang.cbor.value.type = class java.lang.Object
(com.insa.kafka.serializers.yang.cbor.KafkaYangCborSchemaDeserializerConfig:369)
Deserialize: SchemaID:1
Validating at deserialization
TODO: ALEX validate jsonNode against Module
Schema Context:1
Modules: insa-custom
Json node: {"data": {"insa-custom:insa-container": {"computer": "computer", "router": 234}}} -- false
{"data": {"insa-custom:insa-container": {"computer": "computer", "router": 234}}}

```

T2: consumer

Message
deserialized and
printed as JSON

partition <4>, timestamp <15>
Timestamp is the number of milliseconds
since epoch UTC.
Requires broker >= 0.10.0.0 and librdkafka.

```

kcat -b localhost:9092 -t yang.tests -o beginning -C
% Reached end of topic yang.tests [0] at offset 2

hexdump -C file.out
00000000 00 00 00 00 01 bf 64 64 61 74 61 bf 78 1c 69 6e |.....ddata.X.in|
00000010 73 61 2d 66 63 75 73 74 6f 6d 3a 09 6e 73 61 2d |isa-custom:insa-|
00000020 63 6f 6e 74 61 69 66 6e 65 72 bf 68 63 6f 6d 70 |container:hcomp|
00000030 75 74 65 72 68 63 6f 6d 70 75 74 65 72 66 72 6f |uterhcomputerfro|
00000040 75 74 65 72 18 ea ff ff ff 0a 00 00 00 01 bf |uter.....|
00000050 64 64 61 74 61 bf 78 1c 69 6e 73 61 2d 66 63 75 |ddata.X.insa-fcu|
00000060 73 74 6f 6d 3a 09 6e 73 61 2d 63 6f 6e 74 61 69 |stom:insa-contai|
00000070 66 6e 65 72 bf 68 63 6f 6d 70 75 74 65 72 68 63 |computerfouter..|
00000080 6f 6d 70 75 74 65 72 66 72 6f 75 74 65 72 18 ea |....|
00000090 ff ff ff 0a
00000094

```

T3: kcat (kafka
on the wire)

Message encoded
in CBOR on the
wire

```

./bin/schema-registry-start config/schema-registry.properties
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers.checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.SubjectVersionsResource registered in SERVER runtime does not im
cable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.S
d.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers.checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.CompatibilityResource registered in SERVER runtime does not impleme
ble in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schema.registry.rest.resources.CompatibilityResource will be ignored.
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers.checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.ModelResource registered in SERVER runtime does not implement any provider interfaces applica
e SERVER runtime. Due to constraint config
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers.checkProviderRuntime
WARNING: A provider io.confluent.kafka.schema.registry.rest.resources.ModelResource registered in SERVER runtime does not implement any provider interfaces applic
able in the SERVER runtime. Due to constraint config
May 16, 2024 11:57:35 AM org.glassfish.jersey.internal.inject.Providers.checkProviderRuntime
WARNING: A provider io.confluent.dekregistry.rest.resources.DeregisterResource registered in SERVER runtime does not implement any provider interfaces applica
ble in the SERVER runtime. Due to constraint config
ERROR StatusLogger Log4j2 could not find a logging provider. Please add log4j-core to the classpath.
(2024-05-16 11:57:36,271) INFO Started o...
(2024-05-16 11:57:36,282) INFO Started o...
(2024-05-16 11:57:36,297) INFO Started Net...
(2024-05-16 11:57:36,298) INFO Started at 36...
(2024-05-16 11:57:36,299) INFO Schema Registry version: 7.5.1 commitId: fdede4252ab4927d6cd53148d1b5ee99283e6a7 (io.confluent.kafka.schema.registry.rest.SchemaRegistryMain:417)
(2024-05-16 11:57:36,299) INFO Server started, listening for requests... (io.confluent.kafka.schema.registry.rest.SchemaRegistryMain:49)
(2024-05-16 12:00:19,419) INFO Registering new schema: subject yang.tests-value, version null, id null, type YANG, schema size 389 (io.confluent.kafka.schema.registry.rest.resources.SubjectVersionsResource:382)
(2024-05-16 12:00:19,492) INFO 127.0.0.1 -- [16/May/2024:18:00:18 +0000] "POST /subjects/yang.tests-value/versions?normalize=false HTTP/1.1" 200 0 "-" Java/11.0.19 217 (io.confluent.rest-utils.requests:62)
(2024-05-16 12:00:20,102) INFO 127.0.0.1 -- [16/May/2024:18:00:20 +0000] "GET /schemas/ids/1?fetchMaxId=false&subject=yang.tests-value HTTP/1.1" 200 436 "-" Java/11.0.19 13 (io.confluent.rest-utils.requests:62)

```

T4: Schema
registry

2 requests to schema registry:

- registration of the YANG (POST)
- retrieval of the YANG (GET)

What's on the wire (Kafka)

- Hexdumping results from kcat

```

kcat -b localhost:9092 -t yang.tests -o beginning -C
% Reached end of topic yang.tests [0] at offset 0
ddata.xinsa-custom:insa-container@hcomputerhcomputerfrouter0000
% Reached end of topic yang.tests [0] at offset 1
^C

kcat -b localhost:9092 -t yang.tests -o beginning -C > onthewire.out
% Reached end of topic yang.tests [0] at offset 1
^C

hexdump -C onthewire.out
00000000 00 00 00 00 01 bf 64 64 61 74 61 bf 78 1a 69 6e |.....ddata.x.in|
00000010 73 61 2d 63 75 73 74 6f 6d 3a 69 6e 73 61 2d 63 |sa-custom:insa-c|
00000020 6f 6e 74 61 69 6e 65 72 bf 68 63 6f 6d 70 75 74 |ontainer.hcomput|
00000030 65 72 68 63 6f 6d 70 75 74 65 72 66 72 6f 75 74 |erhcomputerfrout|
00000040 65 72 18 ea ff ff ff 0a |er.....|
00000048
```

Magic_byte

Schema_id

Limitations

- YangKit bugs (Vivek fixed most of them)
 - Details: Frank is also pushing some code to YangKit

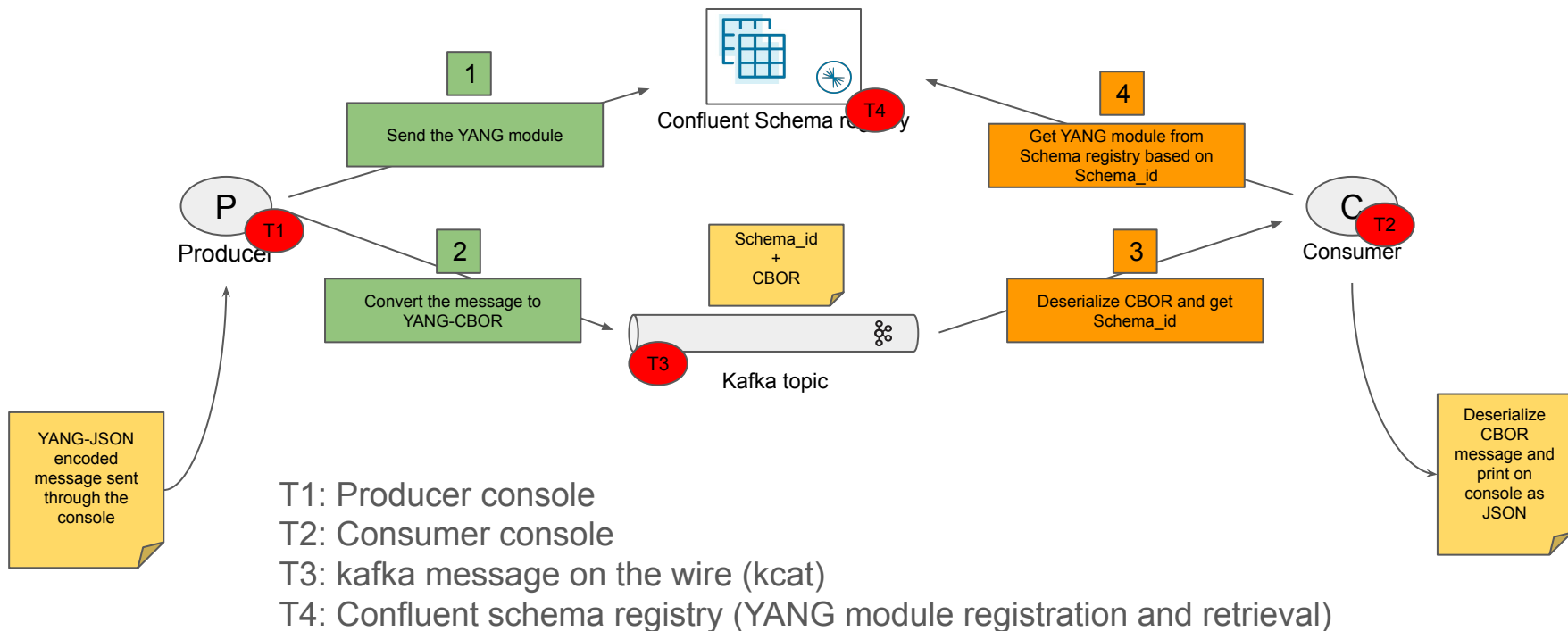
What's next?

- Finish JAVA client (instead of console) to push/receive messages
 - Allowing registering multiple yang modules
 - <https://github.com/network-analytics/schema-registry-samples>
 - Planned for Vivek

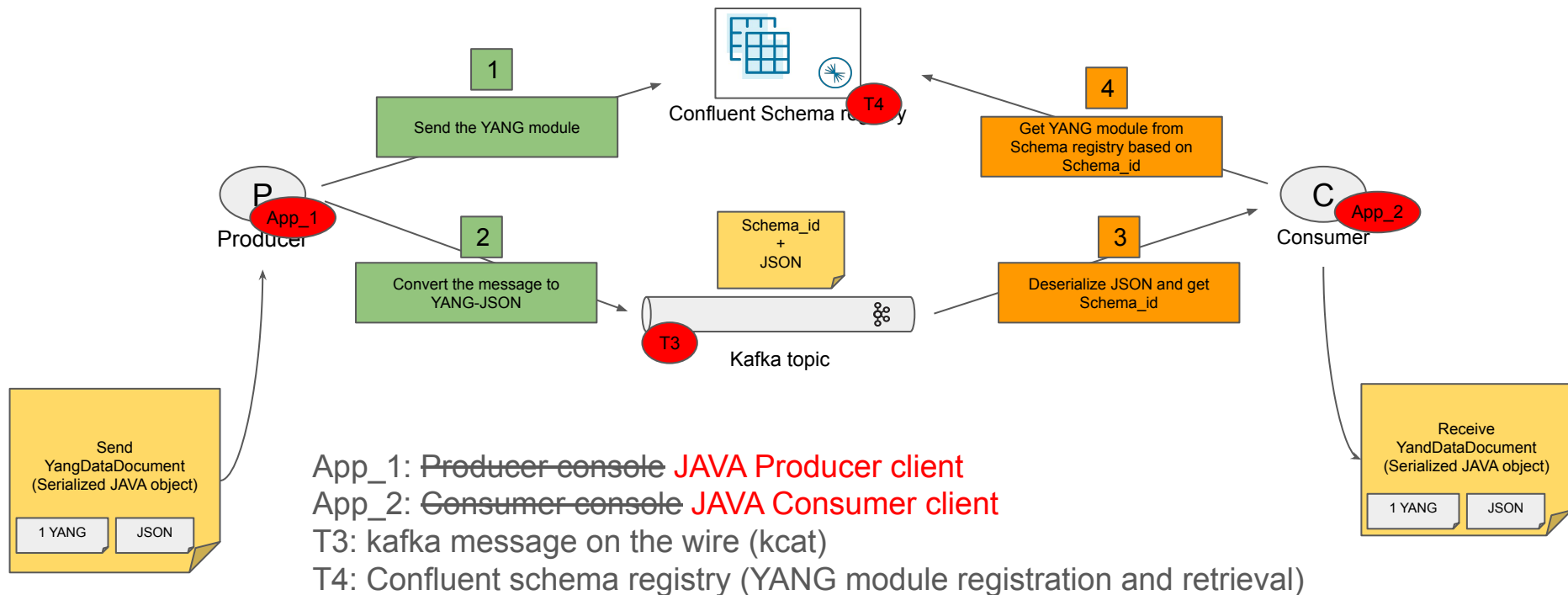
Schema registry - Status (05/07/2024) [Phase 2]

- Standalone JAVA producer and consumer clients
 - Simulating what a developer will use to produce/consume YANG encoded data
 - **YangDataDocument** as interface rather than JsonNode
 - Producer validates message $\longleftrightarrow$ 1 YANG module
 -
- Encodings
 - YANG JSON
 - YANG CBOR (Will be implemented when all the JSON-encoding architecture is completed)

What did we have? [Phase 1]



What do we have now? [Phase 2]



Message validation

→ Producer validation

- ◆ message $\longleftrightarrow$ 1 local YANG module [setting validate at schema registry]
 - The YANG module is given at the same time as the message
- ◆ message $\longleftrightarrow$ 1 local YANG + “imported” YANG modules registered at the schema registry
- ◆ message $\longleftrightarrow$ N local YANG modules [WIP]

→ Consumer validation

- ◆ message $\longleftrightarrow$ schema registry entry identifier
 - The yang schemas are fetched from the registry
 - The message is validated
- ◆ Assumes that the schema registry has been prepared by the “producer” (orchestrator)

YANG Producer Client

```
producerConfig.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9092");
producerConfig.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, org.apache.kafka.common.serialization.StringSerializer.class.getName());
producerConfig.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaYangJsonSchemaSerializer.class.getName()); 1
producerConfig.setProperty(KafkaYangJsonSchemaSerializerConfig.YANG_JSON_FAIL_INVALID_SCHEMA, "true"); 2
producerConfig.setProperty(KafkaYangJsonSchemaSerializerConfig.VALUE_SUBJECT_NAME_STRATEGY, RecordNameStrategy.class.getName()); 3

producerConfig.setProperty("schema.registry.url", "http://127.0.0.1:8081");

KafkaProducer<String, YangDataDocument> producer = new KafkaProducer<>(producerConfig);

YangSchemaContext schemaContext = YangYinParser.parse(Producer.class.getClassLoader().getResource(name: "json/test.yang").getFile()); 4
schemaContext.validate();
JsonNode jsonNode = new ObjectMapper().readTree(new File(Producer.class.getClassLoader().getResource(name: "json/valid.json").getFile()));
ValidatorResultBuilder validatorResultBuilder = new ValidatorResultBuilder();
YangDataDocument doc = new YangDataParser(jsonNode, schemaContext, isOnlyConfig: false).parse(validatorResultBuilder);
doc.validate();

String key = "key1";
String topic = "yang.tests";

ProducerRecord<String, YangDataDocument> record = new ProducerRecord<>(topic, key, doc); 5
try {
    producer.send(record);
} catch (SerializationException e) {
    System.out.println(e.getMessage());
} finally {
    producer.flush();
    producer.close();
}
```

YANG Consumer Client

```
consumerConfig.setProperty(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9092");
consumerConfig.setProperty(ConsumerConfig.GROUP_ID_CONFIG, "test");
consumerConfig.setProperty(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, org.apache.kafka.common.serialization.StringDeserializer.class.getName());
consumerConfig.setProperty(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, KafkaYangJsonSchemaDeserializer.class.getName()); 1
consumerConfig.setProperty(KafkaYangJsonSchemaDeserializerConfig.YANG_JSON_FAIL_INVALID_SCHEMA, "true"); 2
consumerConfig.setProperty(KafkaYangJsonSchemaDeserializerConfig.VALUE_SUBJECT_NAME_STRATEGY, RecordNameStrategy.class.getName()); 3
consumerConfig.setProperty("schema.registry.url", "http://127.0.0.1:8081");
consumerConfig.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

4

KafkaConsumer<String, YangDataDocument> consumer = new KafkaConsumer<>(consumerConfig);
String topic = "yang.tests";

consumer.subscribe(Collections.singletonList(topic));

while (true) {
    try {
        5
        ConsumerRecords<String, YangDataDocument> records = consumer.poll(Duration.ofMillis(100));
        for (ConsumerRecord<String, YangDataDocument> r : records) {
            ObjectMapper mapper = new ObjectMapper();
            JsonNode jsonNode;
            jsonNode = mapper.readTree(r.value().getDocString());
            System.out.println("Clé : " + r.key() + ", Valeur : " + jsonNode + ", Offset : " + r.offset());
        }
    } catch (RecordDeserializationException e) {
        System.out.println("Error during deserialization : message is ignored");
        consumer.seek(e.topicPartition(), e.offset() + 1L);
    } catch (JsonProcessingException ignored) {
        System.out.println("Error reading json");
    }
}
```

YANG Producer Client

```
2024-06-27 19:14:13 [ INFO] - org.apache.kafka.clients.Metadata -Metadata.java(287) -
TODO_ALEX: AM I serializing?
2024-06-27 19:14:13 [ INFO] - org.apache.kafka.clients.producer.internals.TransactionM
Schema is: com.swisscom.kafka.schemaregistry.yang.YangSchema@74fec8cc
2024-06-27 19:14:14 [ INFO] - org.apache.kafka.clients.producer.KafkaProducer -KafkaPro
2024-06-27 19:14:14 [ INFO] - org.apache.kafka.common.metrics.Metrics -Metrics.java(
2024-06-27 19:14:14 [ INFO] - org.apache.kafka.common.metrics.Metrics -Metrics.java(
2024-06-27 19:14:14 [ INFO] - org.apache.kafka.common.metrics.Metrics -Metrics.java(
2024-06-27 19:14:14 [ INFO] - org.apache.kafka.common.utils.AppInfoParser -AppInfoPar
send -> {"data":{"insa-custom:insa-container":{"computer":"computer","router":15}}}
finish !

Process finished with exit code 0
```

YANG Consumer Client

```
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -AbstractCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -AbstractCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -AbstractCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -AbstractCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -AbstractCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -ConsumerCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -AbstractCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -ConsumerCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -ConsumerCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.ConsumerCoordinator -ConsumerCoordi
2024-06-27 19:13:45 [ INFO] - org.apache.kafka.clients.consumer.internals.SubscriptionState -SubscriptionStat
Clé : key1, Valeur : {"data":{"insa-custom:insa-container":{"computer":"computer","router":15}}}, Offset : 0
```

Current use cases

1. Validation at the Producer

- a. Message sent after valid validation
- b. Message sent even if the message is not valid

2. Validation at the Consumer

- a. Message validated (using the Schema_id from the message)

Limitations

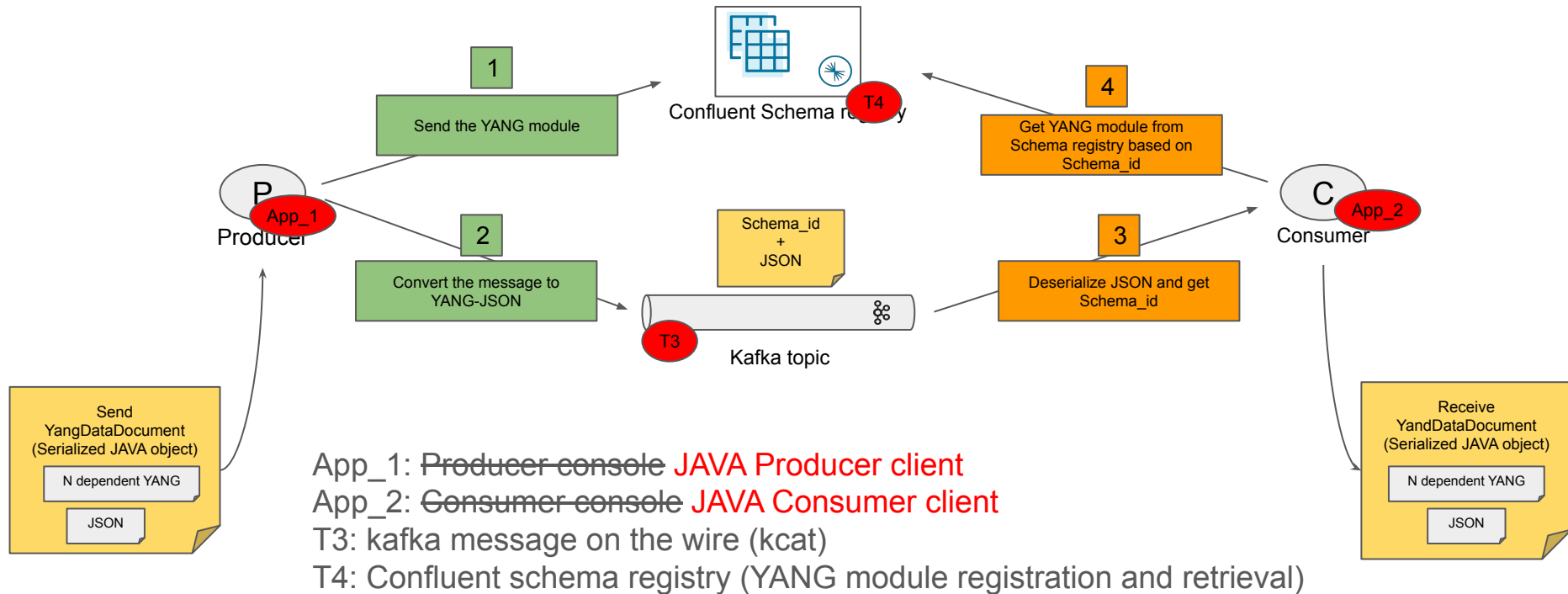
- One YANG module as schema
- “data” envelope still present in YangKit

What's next?

- Registration of multiple YANG modules
- N dependent YANG (Phase 3)
- N independent YANG modules (Phase 4)

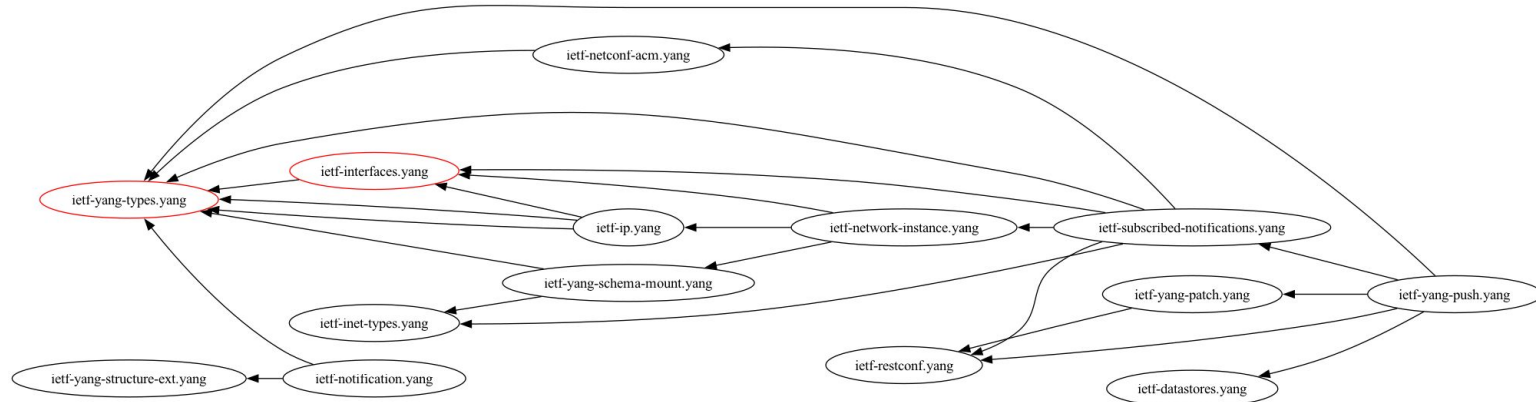
→ Ping Frank? (Vivek noticed some YangKit activity again)

Next steps [Phase 3]

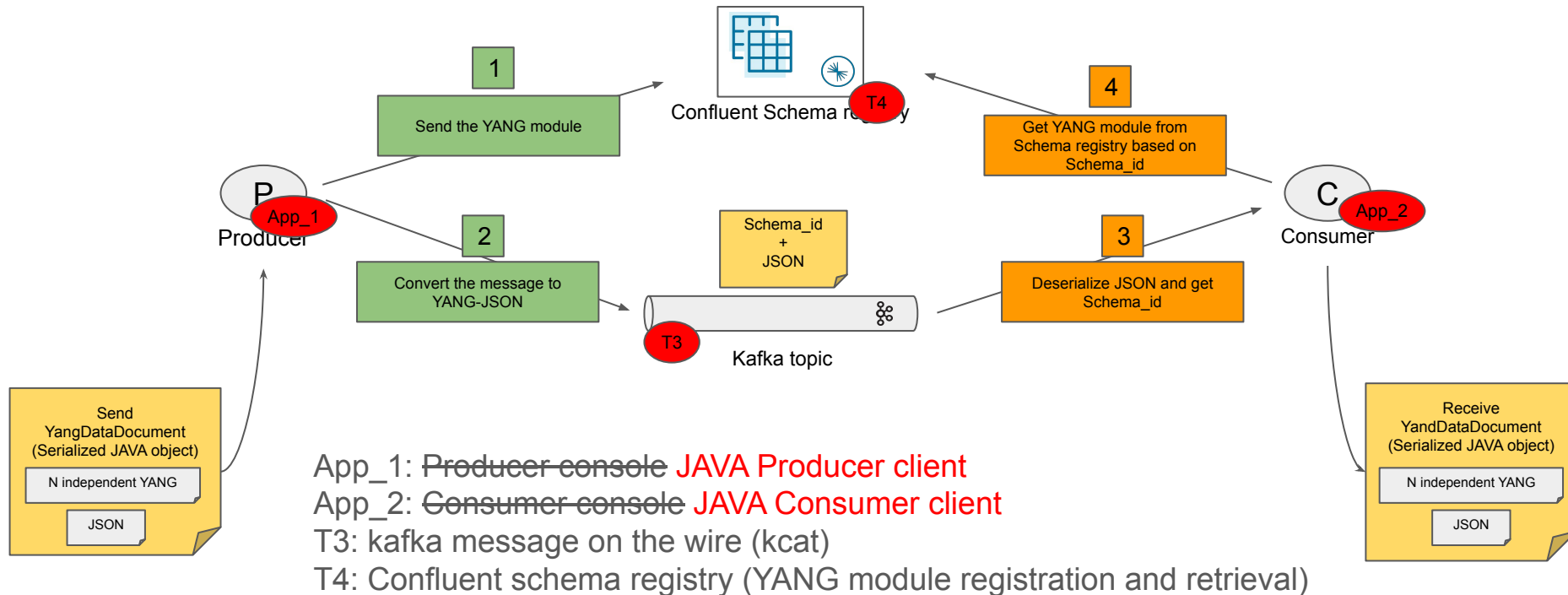


N Dependent YANG = 1 single YANG tree

Example: ietf-yang-push.yang



Next steps [Phase 4]

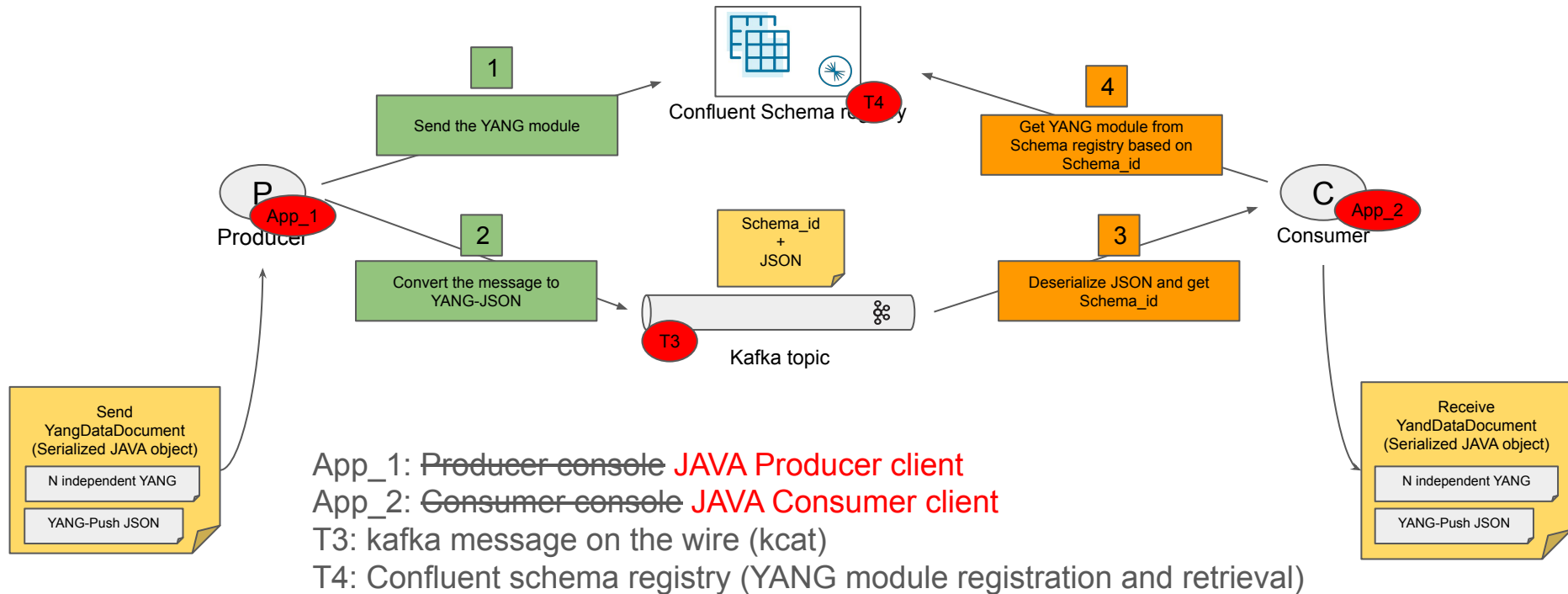


N independent YANG = more than one YANG tree

Example: ietf-notification.yang & ietf-yang-push.yang



What do we have now? [Phase 5] → YANG-Push messages



Yangkit validation gaps

- Type validation
- Mandatory
- Lists
- Unknown elements

Validating YANG-JSON data: Type validation

Leaf **computer**:

- YANG definition: string
- Content in the data: integer

Expected: **Throw an error**

Reality: Validator doesn't throw any error



```
{
  "data": {
    "insa-custom:insa-container": {
      "computer": 1,
      "router": 234
    }
  }
}
```



```
description
  "insa-test YANG module.";

revision 2023-09-05 {
  description "Initial version.";
}

container insa-container {
  config false;
  leaf computer {
    type string;
    mandatory true;
    description "computer";
  }
  leaf router {
    type uint8;
    description "router";
  }
}
```


Validating YANG-JSON data: Mandatory leaves

Leaf **computer**:

- YANG definition: **mandatory**
- Content in the data: **not present**

Expected: **Throw an error**

Reality: Validator doesn't throw any error



```
{
  "data": {
    "insa-custom:insa-container": {
      "router": 234
    }
  }
}
```



```
description
  "insa-test YANG module.";

revision 2023-09-05 {
  description "Initial version.";
}

container insa-container {
  config false;
  leaf computer {
    type string;
    mandatory true;
    description "computer";
  }
  leaf router {
    type uint8;
    description "router";
  }
}
```

Validating YANG-JSON data: Lists (missing keys)

Leaf **computer**:

- YANG definition: Key of a list
- Content in the data: not present

Expected: Throw an error

Reality: Validator throws a “**missing-element**” error

```
{
  "data": {
    "insa-custom:insa-container": [{
      "router": 234
    }]
  }
}
```



```
list insa-container {
  key computer;
  config false;
  leaf computer {
    type string;
    mandatory true;
    description "computer";
  }
  leaf router {
    type uint8;
    description "router";
  }
}
```



Validating YANG-JSON data: Unknown elements

Leaf `invalid_key`:

- YANG definition: **Not defined**
- Content in the data: **Present**

Expected: **Throw an error**

Result: Validator throws an “**unknown element**” error



```
{
  "data": {
    "insa-custom:insa-container": {
      "computer": "computer",
      "router": 234,
      "invalid_key": "hey"
    }
  }
}
```

```
description
  "insa-test YANG module.";

revision 2023-09-05 {
  description "Initial version.";
}

container insa-container {
  config false;
  leaf computer {
    type string;
    mandatory true;
    description "computer";
  }
  leaf router {
    type uint8;
    description "router";
  }
}
```

Validating YANG push Notification

- Accepts a YANG module defining a structure
- Can separate YANG push header from the payload
- Data Validation: similar to YANG data

Corner cases to have in mind

XPath is not managed by Yangkit

- YANG push subscription need to validate the data defined by the XPath
- The rest of the YANG module, the data should not be present

→ Yangkit approach: validate against a general YANG tree defined by **SchemaContext**

Corner cases to have in mind

Xpath=/example:insa-container

Xpath=/example:test-container

```
revision 2019-09-09 {  
  description "Initial version.";  
}  
  
container insa-container {  
  config false;  
  leaf computer {  
    type string;  
    description "computer";  
  }  
  leaf router {  
    type uint8;  
    description "router";  
  }  
}  
  
container test-container {  
  config false;  
  leaf my-id {  
    type string;  
    mandatory true;  
    description "identifier";  
  }  
  leaf subscription {  
    type string;  
    description "subscription";  
  }  
}
```

Mandatory field

Corner cases to have in mind

Subscription to

- Xpath=/example:insa-container

Yangkit default behavior (Checked with Frank):

- Data not having /test-container/my-id should **fail**

```
{
  "data": {
    "insa-custom:insa-container": {
      "computer": "computer",
      "router": 234
    }
  }
}
```

```
revision 2019-09-09 {
  description "Initial version.";
}
```

```
container insa-container {
  config false;
  leaf computer {
    type string;
    description "computer";
  }
  leaf router {
    type uint8;
    description "router";
  }
}

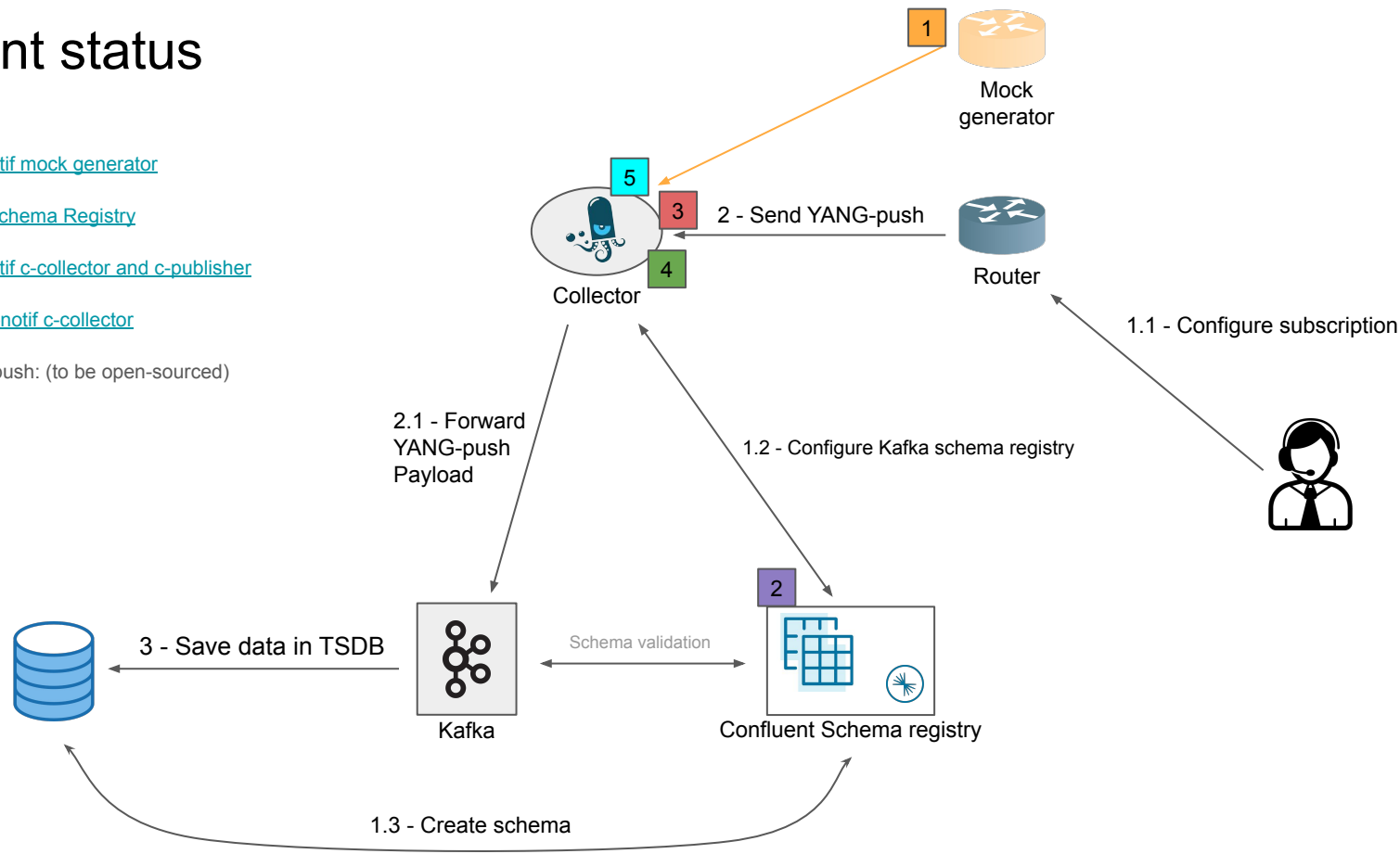
container test-container {
  config false;
  leaf my-id {
    type string;
    mandatory true;
    description "identifier";
  }
  leaf subscription {
    type string;
    description "subscription";
  }
}
```

Missing pieces

libyangserdes

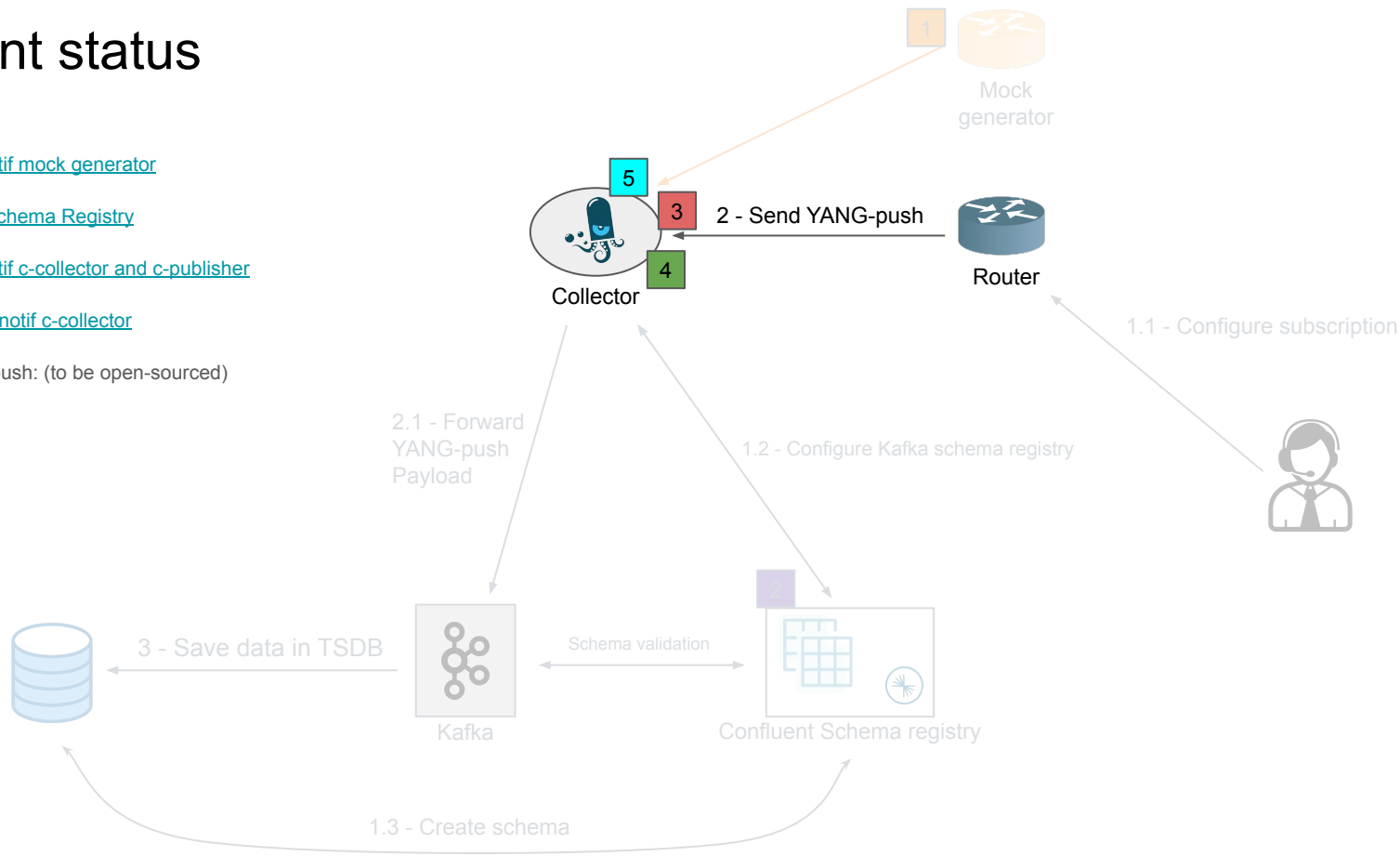
Current status

- 1 [UDP-notif mock generator](#)
- 2 [Kafka Schema Registry](#)
- 3 [UDP-notif c-collector and c-publisher](#)
- 4 [HTTPS-notif c-collector](#)
- 5 libyangpush: (to be open-sourced)

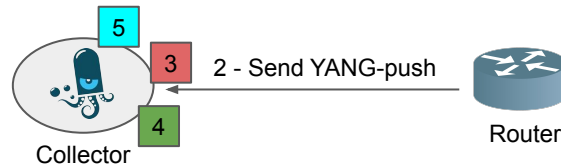


Current status

- 1 [UDP-notif mock generator](#)
- 2 [Kafka Schema Registry](#)
- 3 [UDP-notif c-collector and c-publisher](#)
- 4 [HTTPS-notif c-collector](#)
- 5 libyangpush: (to be open-sourced)



Current status: router - collector



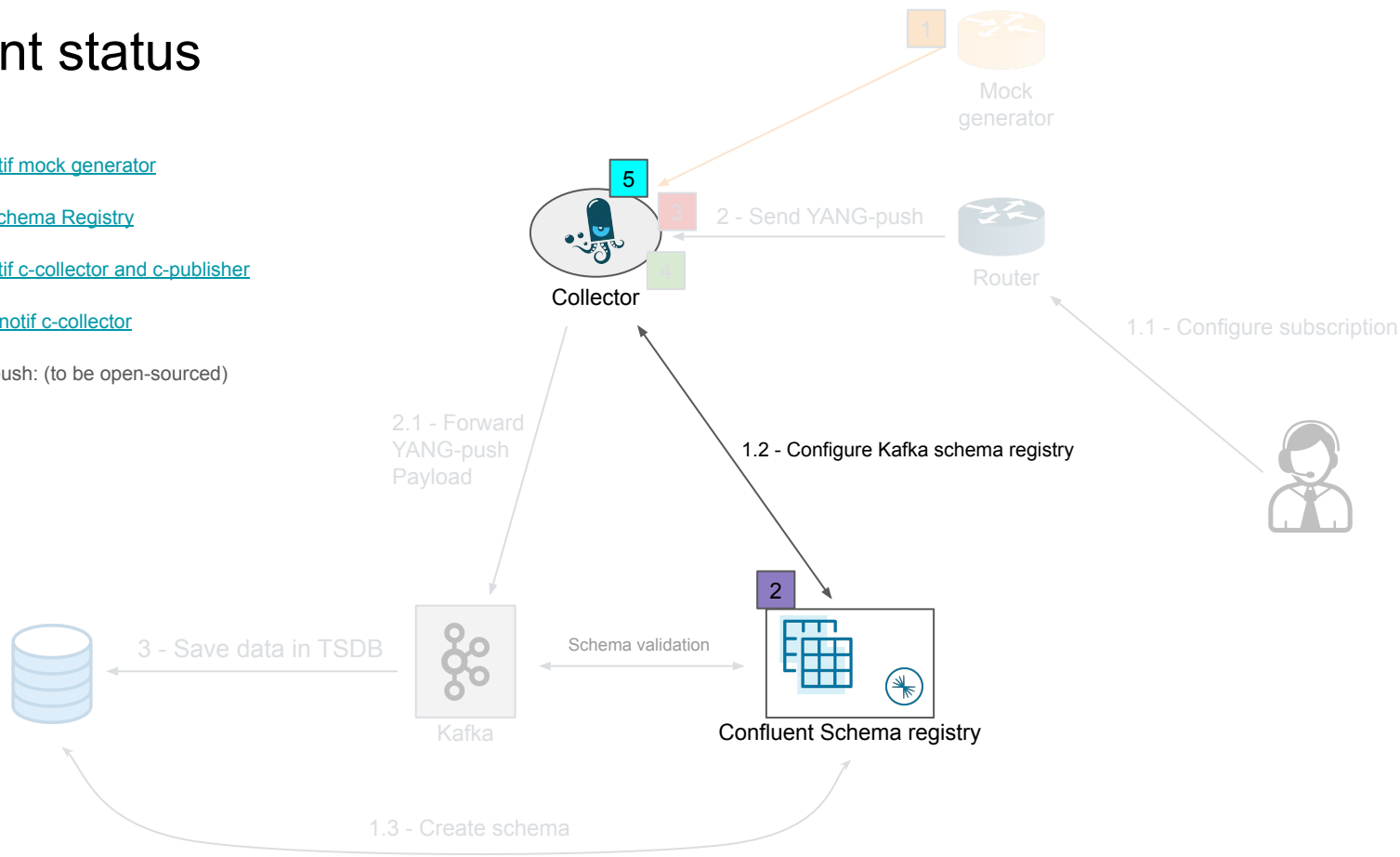
- Router
 - able to craft UDP-notif messages using: [UDP-notif c-collector and c-publisher](#)
- Collector
 - able to collect UDP-notif messages using: [UDP-notif c-collector and c-publisher](#)
 - able to collect HTTPS-notif messages using: [HTTPS-notif c-collector](#)
 - able to get YANG modules dependencies using: libyangpush (to be open-sourced)

YANG-JSON and YANG-CBOR encoding is generated at Router independently from the transport protocol.

*Current Transports already support **YANG-JSON** and **YANG-CBOR** MediaType*

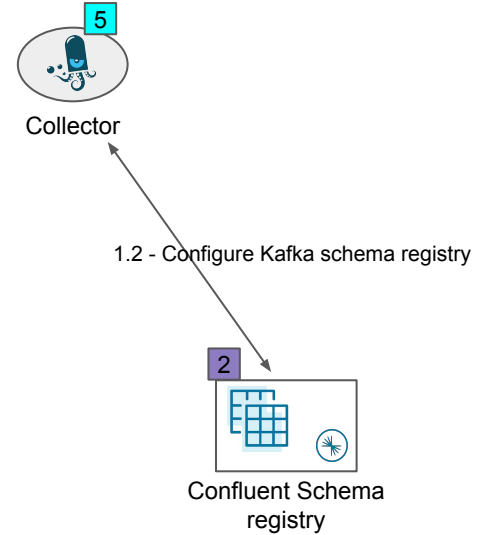
Current status

- 1 [UDP-notif mock generator](#)
- 2 [Kafka Schema Registry](#)
- 3 [UDP-notif c-collector and c-publisher](#)
- 4 [HTTPS-notif c-collector](#)
- 5 libyangpush: (to be open-sourced)



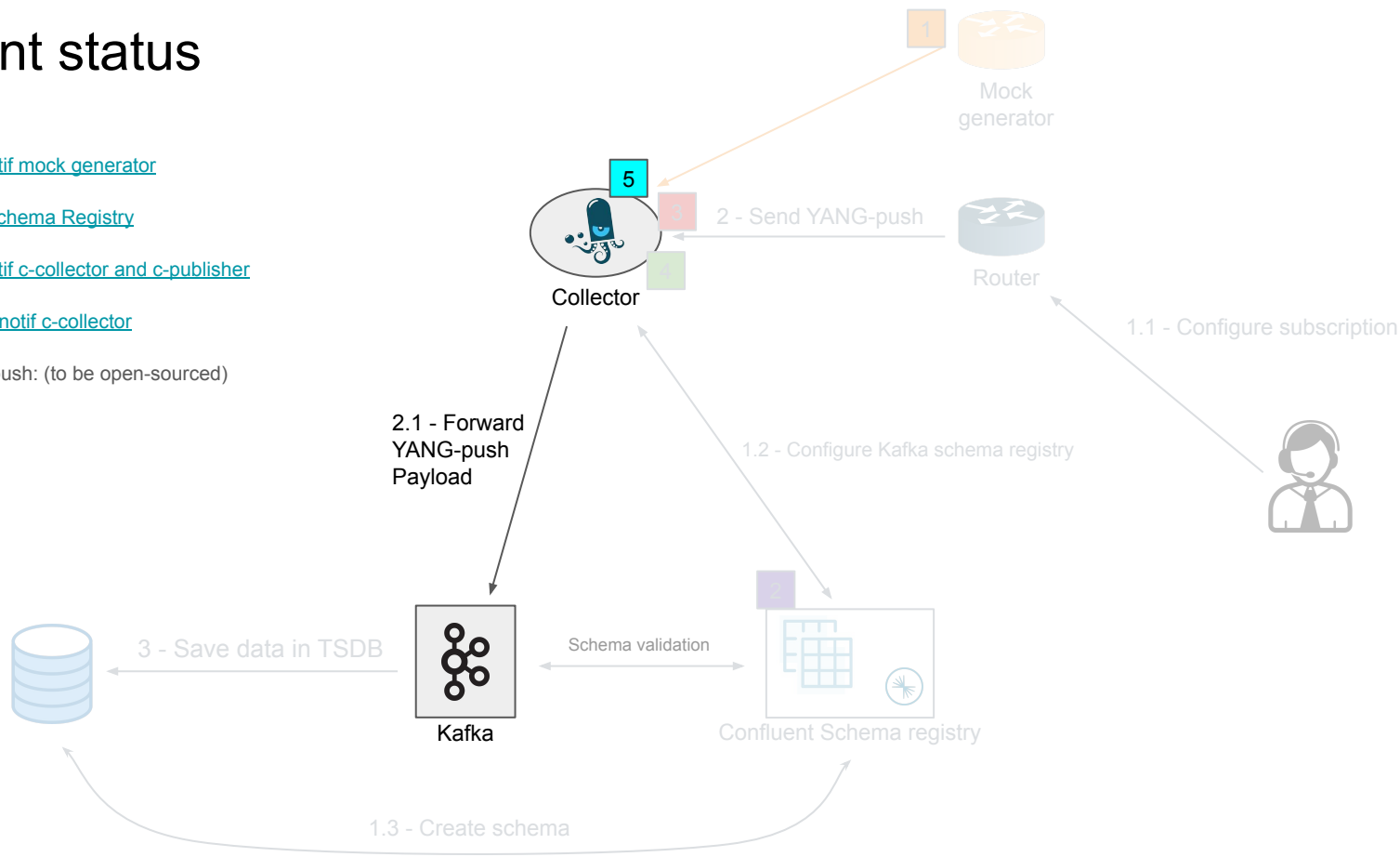
Current status: collector - schema registry

- Collector
 - when the YANG push message is received, we are able to get all YANG modules from the router using `libyangpush`
 - Generate registration to Schema Registry using `libyangpush`
- Schema registry
 - Support **YANG** as a schema type
 - *WIP: one subject defines one YANG module or one Schema Context*



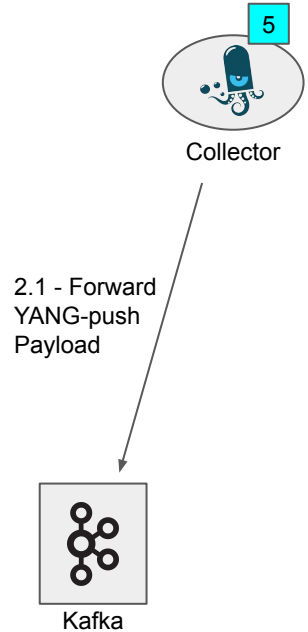
Current status

- 1 [UDP-notif mock generator](#)
- 2 [Kafka Schema Registry](#)
- 3 [UDP-notif c-collector and c-publisher](#)
- 4 [HTTPS-notif c-collector](#)
- 5 libyangpush: (to be open-sourced)



Current status: collector - Kafka topic

- Collector
 - Once the collector got the Schema_id serialize the message to Kafka using **libyangserdes** (Craft Kafka message with MAGICBYTE and Schema_id)
- Kafka
 - is able to receive JSON or CBOR messages

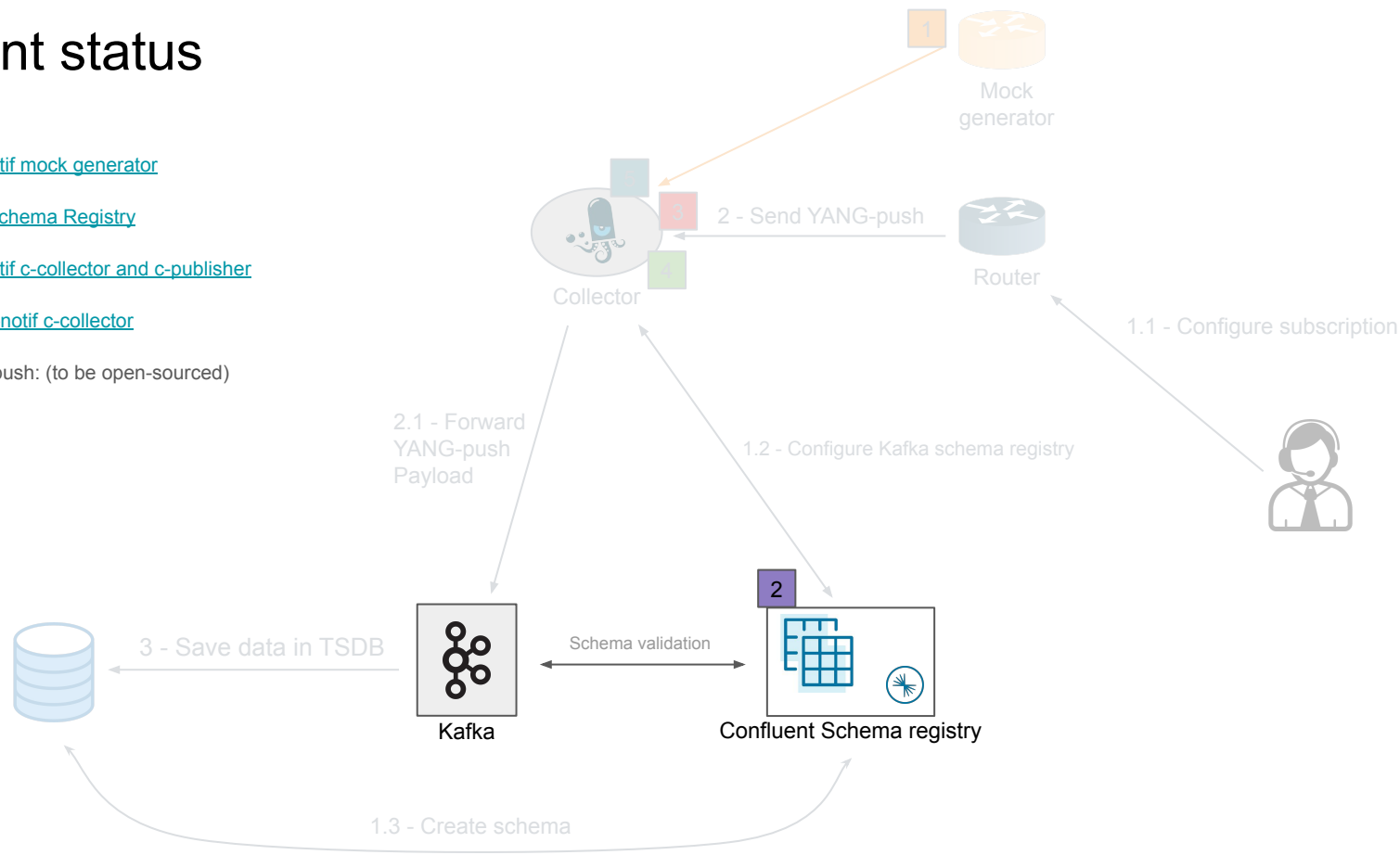


libyangserdes

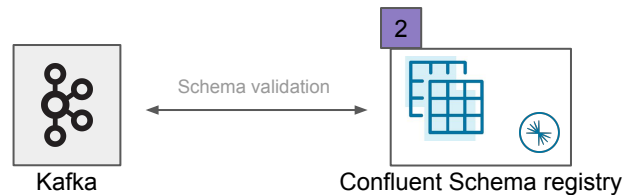
- Client uses `libyangpush` to register all the YANG modules to Schema Registry
- Client uses `libyangserdes` to craft the kafka serialised message including the ***MAGICBYTE*** and the ***Schema_id***
- `libyangserdes` must not modify the content of the message

Current status

- 1 [UDP-notif mock generator](#)
- 2 [Kafka Schema Registry](#)
- 3 [UDP-notif c-collector and c-publisher](#)
- 4 [HTTPS-notif c-collector](#)
- 5 libyangpush: (to be open-sourced)



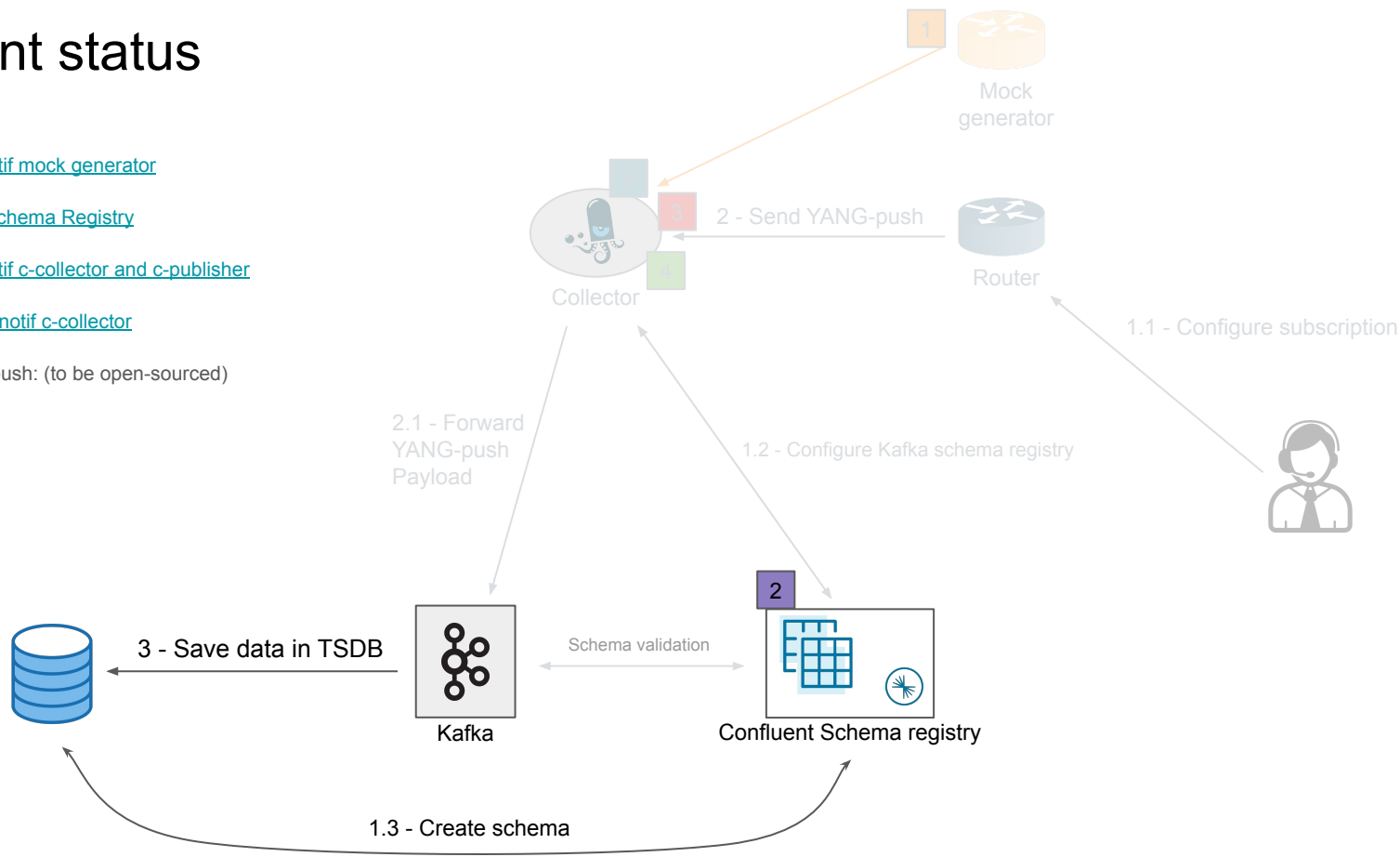
Current status: Kafka - Schema Registry



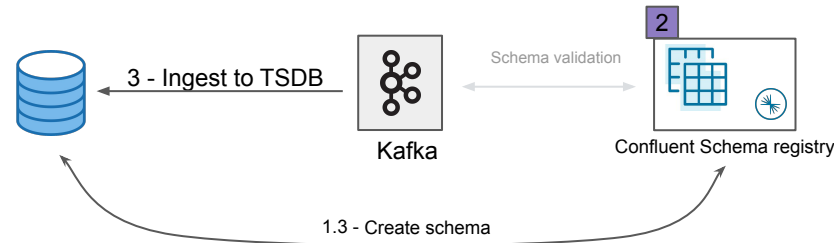
- Kafka
 - is able to get the Schema_id from the message
 - is able to request the schema from the Schema Registry and validate the content
 - WIP: YANG validation with Yangkit
- Confluent Schema Registry
 - is able to provide the schema / schema context
 - WIP: Current discussions if current API need to be modified to accommodate YANG subjects

Current status

- 1 [UDP-notif mock generator](#)
- 2 [Kafka Schema Registry](#)
- 3 [UDP-notif c-collector and c-publisher](#)
- 4 [HTTPS-notif c-collector](#)
- 5 libyangpush: (to be open-sourced)



Current status: TSDB ingestion



- TSDB

- Druid uses Kafka connect to create/ingest data to the database
- Missing: YANG connector (part of Druid)

- Kafka

- is able to provide the schema / schema context
- WIP: Current discussions if current API need to be modified to accommodate YANG subjects

- Confluent Schema registry

- is able to provide the schema / schema context
- WIP: Current discussions if current API need to be modified to accommodate YANG subjects